# Music Genre Classification

## A Modular Research Pipeline on GTZAN with Tabular Features and End-to-End Mel Spectrogram Models

GROUP 5

Nguyen Trung Hieu - 202416689
Tran Dang Minh - 202416720
Vu Ha Anh Duc - 202416675
Dinh Gia Bao - 202416666
Trinh Hai Phong - 202416732

---

# Abstract

This project implements a modular, reproducible pipeline for **music genre classification** on a GTZAN mirror stored under `Data/`. Two modeling tracks are supported: (i) **tabular learning** on pre-extracted 3-second segment features from `Data/features_3_sec.csv` (58-dimensional input after dropping identifier columns), and (ii) **end-to-end neural learning** from raw audio by extracting **mel spectrogram** segments (3 seconds, 128 mel bins) and training deep models (CNN/LSTM). The repository includes stage-wise caching, logging, checkpointing, evaluation reports, and extensive visualizations (audio waveforms, STFT/mel/MFCC/chroma/spectral contrast/tempograms/HPSS, plus embeddings and correlation analysis).

A critical audit of the original tabular split ( `Data/splits/split_v1.csv` ) revealed severe **track-level leakage**: **966/1000 original tracks** had segments appearing in more than one split (train/val/test), inflating reported accuracy. This has been **corrected**: both tracks now use a **group-level split** ( `make_group_splits_from_filenames` ) that guarantees all segments of a given 30-second track fall into exactly one split. The corrected tabular split is saved as `Data/splits/split_by_track_v2.csv` . The mel track had already used track-disjoint group splits and required no change. Additionally, two new data augmentation strategies have been added to address genre overlap ( `Mixup` ) and domain shift ( `SpecAugment` ), with configuration under `data.augmentation` in `configs/config.yaml` . Results under the corrected protocol are pending re-run.

---

# 1. Introduction

**Music Genre Classification (MGC)** aims to assign a genre label (e.g., blues, classical, hiphop) to an audio recording. It is a canonical task in **Music Information Retrieval (MIR)**, often used to study audio representation learning, signal processing features (STFT, Mel, MFCC), and supervised classification.

**Why it matters.** Genre recognition supports recommendation systems, music cataloging, playlist generation, and metadata enrichment. Beyond genre, similar pipelines extend naturally to mood, instrument, artist classification, and tagging.

**Challenges.** - **Inter-class overlap:** many genres share instrumentation or rhythmic patterns (e.g., rock vs. metal). - **Intra-class variability:** genre categories are broad; production style and recording quality vary. - **Temporal structure:** music is non-stationary; short segments may not capture global song identity. - **Evaluation pitfalls:** segmenting tracks and splitting segments naively introduces **data leakage** when segments from the same track appear in train and test.

This repository explicitly focuses on building a **stage-based research pipeline** with caching, reproducibility, visualization, and a “model zoo” spanning classical ML and neural methods.

---

# 2. Problem Analysis

MGC is a multi-class classification problem with label space size **C = 10** (confirmed by `Data/features/label_map.json`): `blues, classical, country, disco, hiphop, jazz, metal, pop, reggae, rock`.

Musically, genres often differ along multiple axes: - **Timbre / instrumentation:** MFCC-like spectral envelope patterns often separate classical vs. metal/rock. - **Rhythm / tempo / percussiveness:** hiphop/disco often show strong rhythmic periodicity; classical may be less percussive. - **Harmonic structure:** chroma patterns can be more structured for tonal/harmonic genres; less stable for percussive-heavy segments. - **Spectral brightness:** centroid/rolloff relate to high-frequency content; distorted guitars/hi-hats shift spectral energy upward.

These factors are partially captured by the project's two feature paradigms: - **Tabular summaries** (per-segment statistical aggregates: mean/variance over 3 seconds). - **Time–frequency images/sequences** (mel spectrogram frames over time).

---

# 3. Dataset Description

## 3.1 Repository dataset layout

The repository includes a GTZAN mirror under `Data/` (see `README.md`): - `Data/genres_original/`: raw audio `.wav` files organized by genre. - `Data/features_3_sec.csv`: tabular features for ~3-second segments (row-wise samples). - `Data/features_30_sec.csv`: tabular features for 30-second tracks (track-wise samples). - `Data/images_original/`: spectrogram images (mirror-dependent). - `Data/processed/metadata.csv`: metadata generated by the pipeline `preprocess` stage. - `Data/splits/*`: split definitions used by the pipeline.

## 3.2 Audio metadata audit

From `Data/processed/metadata.csv` (generated by `src/data/preprocessing.py`): - Total raw audio files scanned: **1000** - Readable/OK: **999** - Unreadable/corrupted: **1** (`.../jazz/jazz.00054.wav`, marked unreadable) - Sample rate distribution (OK files): **22050 Hz for all 999 tracks** - Duration statistics (OK files): mean ≈ **30.02 s** with small variance (min ≈ 29.93 s, max ≈ 30.65 s)

**Figure 1.** Raw audio duration distribution (metadata).

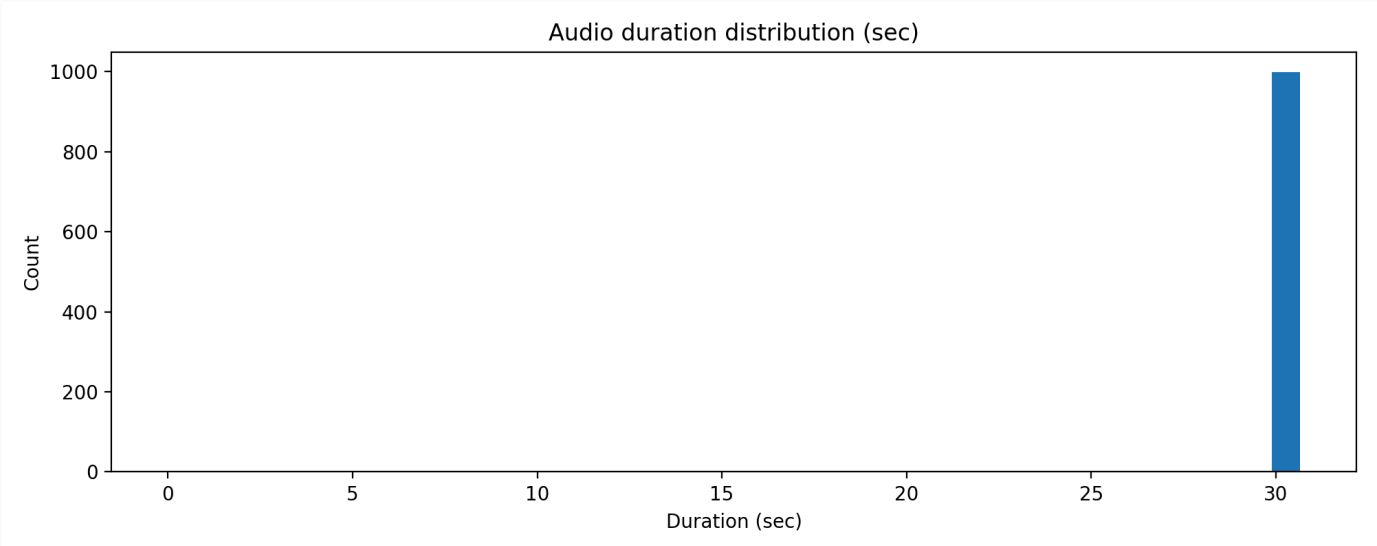



Figure 1: Audio duration distribution (sec) from `Data/processed/metadata.csv`.

**Figure 2.** Raw audio sample rate distribution.

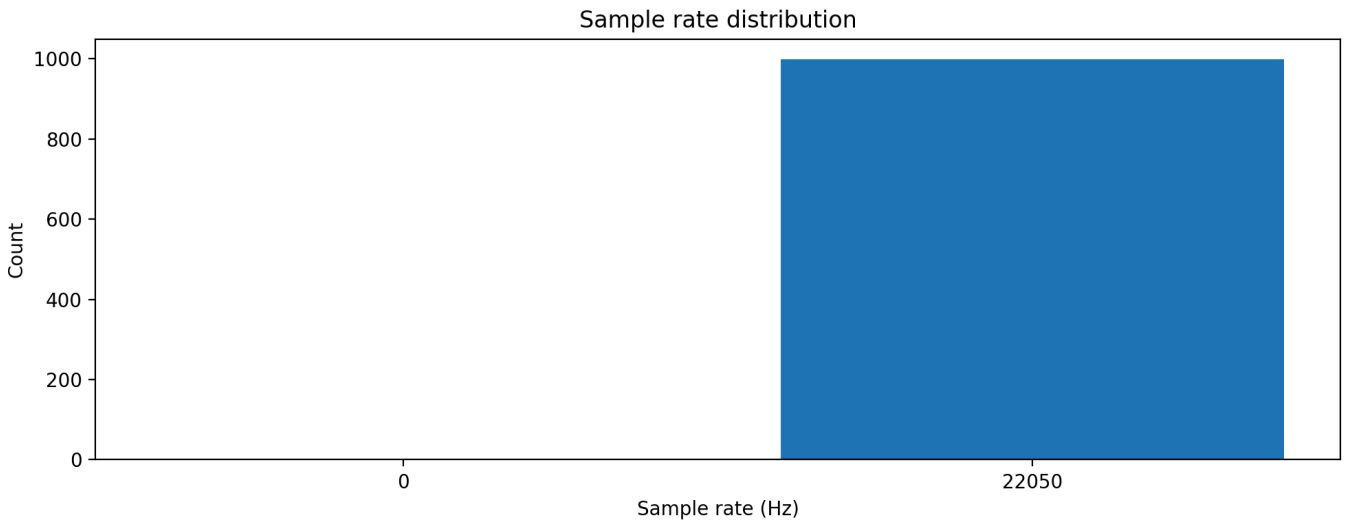



Figure 2: Sample rate distribution (Hz) from `Data/processed/metadata.csv`.

## 3.3 Tabular dataset: feature schema and dimensionality

`Data/features_3_sec.csv` contains 60 columns, including identifiers and labels: - Identifier columns: `filename`, `length` - Label column: `label` - Feature columns: 57 engineered audio features, including: - `chroma_stft_{mean,var}` - `rms_{mean,var}` - `spectral_centroid_{mean,var}`, `spectral_bandwidth_{mean,var}` - `rolloff_{mean,var}` - `zero_crossing_rate_{mean,var}` - `harmony_{mean,var}`, `perceptr_{mean,var}` (percussive component statistics) - `tempo` - `mfcc1..mfcc20_{mean,var}`

The pipeline's tabular cache builder (`src/data/feature_extraction.py`) drops `filename` and `label`, leaving **58 input dimensions** (57 audio features + `length`).

## 3.4 Train/validation/test splits (as shipped in this repo)

### 3.4.1 Tabular split (segment-level, leaky by track)

The provided split file for tabular features is `Data/splits/split_v1.csv` with: - train: **7192** segments - val: **800** segments - test: **1998** segments - total: **9990** segments

Class distribution per split (derived by joining `split_v1.csv` with `features_3_sec.csv` by row index):

| label | train | val | test | TOTAL |
|---|---|---|---|---|
| blues | 720 | 80 | 200 | 1000 |
| classical | 719 | 80 | 199 | 998 |
| country | 718 | 80 | 199 | 997 |
| disco | 719 | 80 | 200 | 999 |

### 3.4.1 Tabular split (segment-level, track-disjoint)

**Split correctness (v2, fixed).** Starting from `feature.yaml` version `split_by_track_v2.csv`, the tabular pipeline uses `src/data/split.py::make_group_splits_from_filenames`. This function: 1. Extracts the track group ID from each filename: `blues.00000.0.wav` → group `blues.00000`. 2. Performs a **stratified group-level split** (via `make_group_splits`), so all segments of a track go to the same split. 3. Persists the result to `Data/splits/split_by_track_v2.csv` (row-indexed, compatible with `build_tabular_cache`).

Result of the v2 group split (seed=42, test=0.2, val=0.1):

| label | train | val | test | TOTAL |
|---|---|---|---|---|
| blues | 720 | 80 | 200 | 1000 |
| classical | 718 | 80 | 200 | 998 |
| country | 727 | 70 | 200 | 997 |
| disco | 729 | 70 | 200 | 999 |
| hiphop | 688 | 110 | 200 | 998 |
| jazz | 740 | 60 | 200 | 1000 |
| metal | 710 | 90 | 200 | 1000 |
| pop | 760 | 40 | 200 | 1000 |
| reggae | 680 | 120 | 200 | 1000 |
| rock | 718 | 80 | 200 | 998 |
| **TOTAL** | **7190** | **800** | **2000** | **9990** |

**Zero track overlap confirmed**: 0 of 1000 tracks appear in more than one split (verified programmatically).

### 3.4.2 Mel split (segment-level, *track-disjoint by construction*)

The mel pipeline extracts 3-second segments directly from raw audio and assigns a **group ID per original 30-second track** (e.g., `blues.00000` ). The split is performed by group ( `src/data/split.py::make_group_splits` ), and the produced cache `Data/features/mel_features_v1.npz` has: - train: **7183** segments - val: **798** segments - test: **2000** segments - total: **9981** segments

The `groups_train/val/test` sets have **zero overlap** (track-disjoint). Test set is perfectly balanced at 200 segments per class.

| | train | val | test | TOTAL |
|---|---|---|---|---|
| blues | 750 | 50 | 200 | 1000 |
| classical | 680 | 118 | 200 | 998 |
| country | 727 | 70 | 200 | 997 |
| disco | 709 | 90 | 200 | 999 |
| hiphop | 708 | 90 | 200 | 998 |
| jazz | 720 | 70 | 200 | 990 |
| metal | 700 | 100 | 200 | 1000 |
| pop | 750 | 50 | 200 | 1000 |
| reggae | 700 | 100 | 200 | 1000 |

| | train | val | test | TOTAL |
|---|---|---|---|---|
| rock | 739 | 60 | 200 | 999 |
| TOTAL | 7183 | 798 | 2000 | 9981 |

---

# 4. Feature Extraction Analysis (with visual evidence)

This project supports two complementary notions of “features”: 1. **Tabular engineered features** from `features_3_sec.csv` (statistical summaries over a 3s window). 2. **Time–frequency representations** computed on-demand from raw audio (STFT, mel, MFCC, chroma, etc.) for analysis and for end-to-end learning.

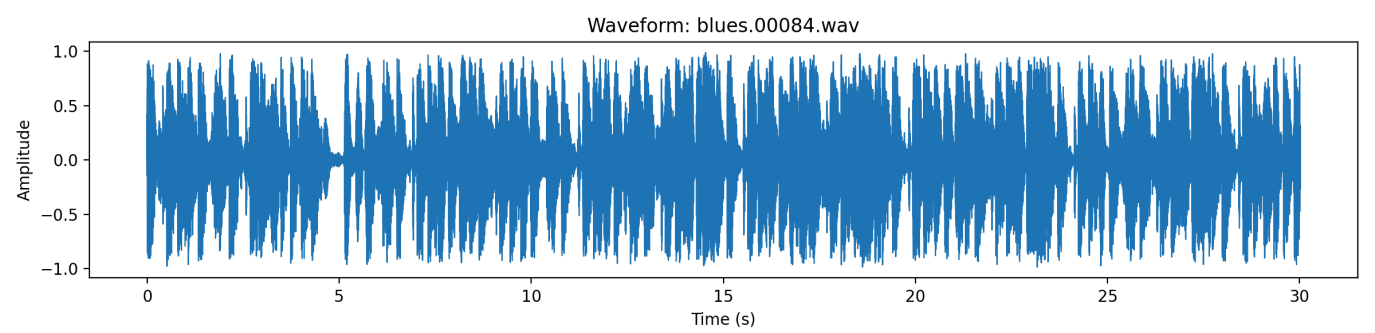
The `visualize` / `visualize-features` stages generate extensive figures under: - `outputs/visualize-features-tabular/figures/` and `outputs/visualize-features-mel/figures/` - `outputs/test-visualize-features/figures/`

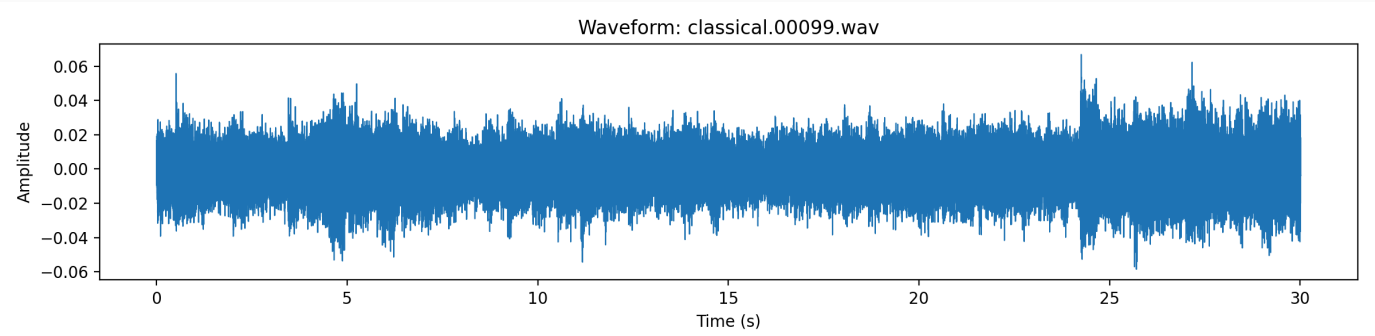
Below, each feature is described mathematically and interpreted using actual generated plots.

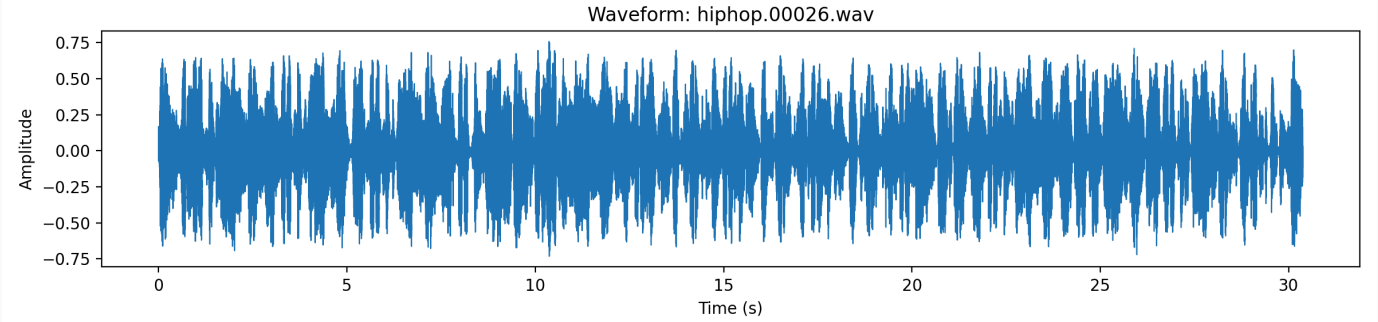
## 4.1 Waveform (time-domain amplitude)

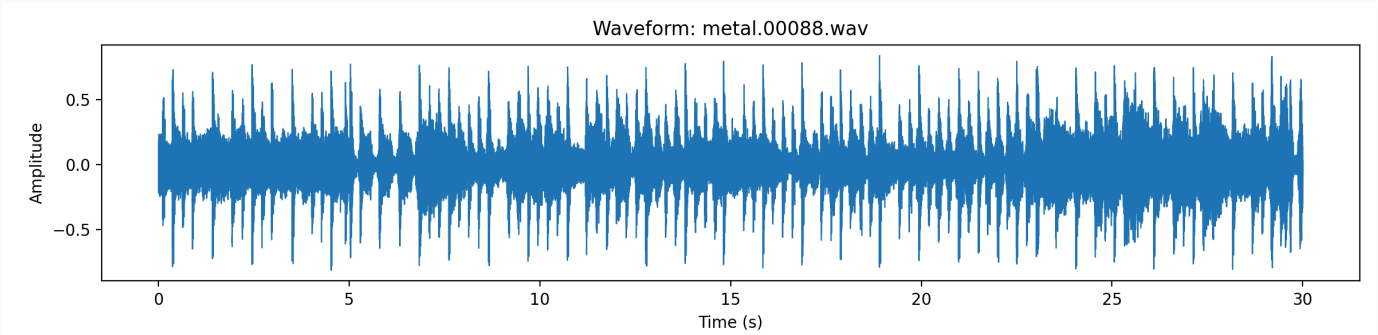
**Meaning.** The waveform is the raw audio signal $x[n]$. While not directly discriminative by itself, it reveals: - amplitude dynamics (compression vs. dynamic range), - silence/structure, - transient density (percussive activity).

**Figure 3–6.** Example waveforms across genres (3-second windows).





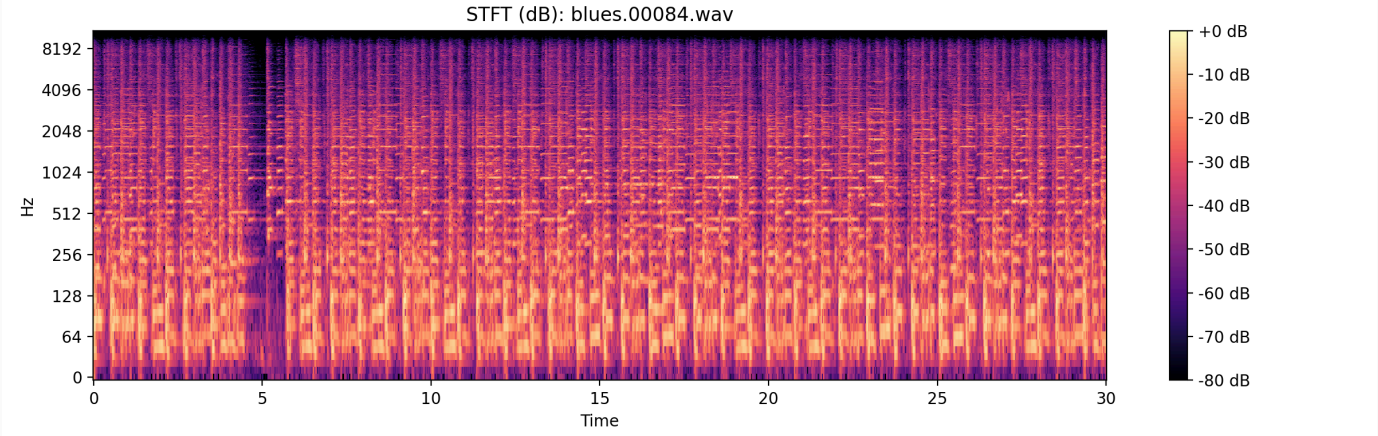

**Interpretation.** Percussive-heavy segments typically show frequent high-amplitude transients; classical often displays larger macro-dynamics (crescendo/decrescendo) with less dense transient activity. These patterns motivate rhythm- and energy-based features (RMS, tempogram, percussive components).

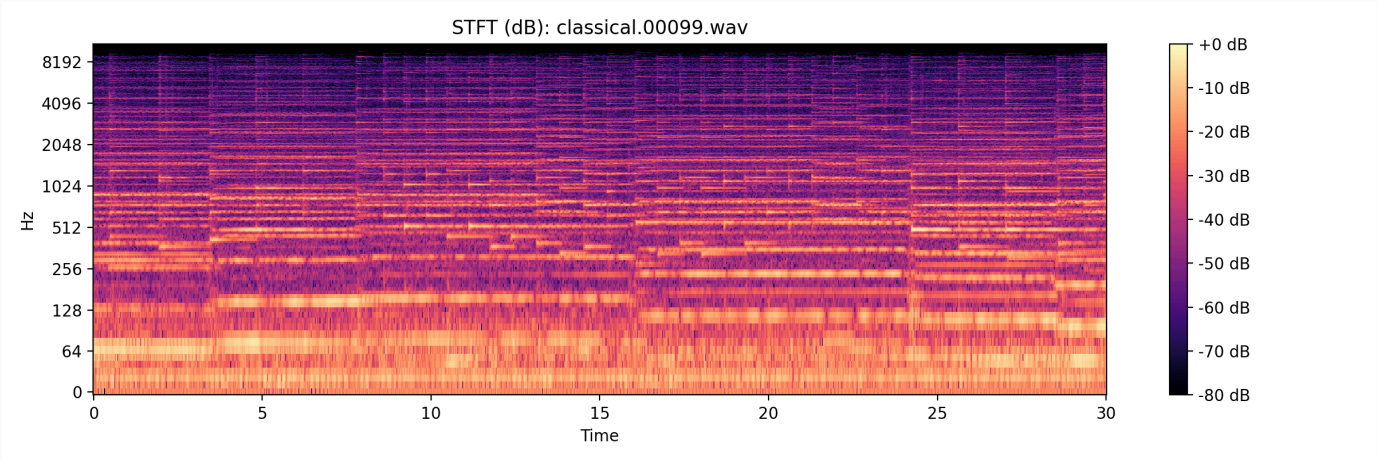
## 4.2 STFT spectrogram (log-frequency, dB)

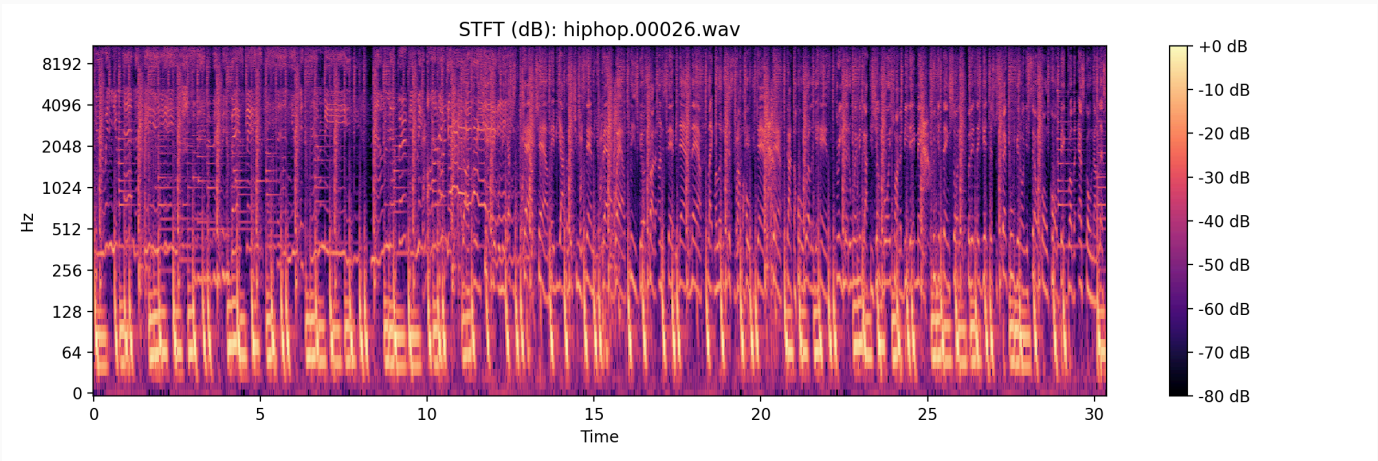
**Meaning.** The Short-Time Fourier Transform (STFT) reveals energy over time and frequency: $X(m, k) = _{n=0}^{N-1} x[n+mH]w[n]e^{-j 2kn/N}, S(m,k)=|X(m,k)|$ The plots use `src/visualization/spectrogram.py::plot_stft` with default `n_fft=2048` , `hop_length=512` .

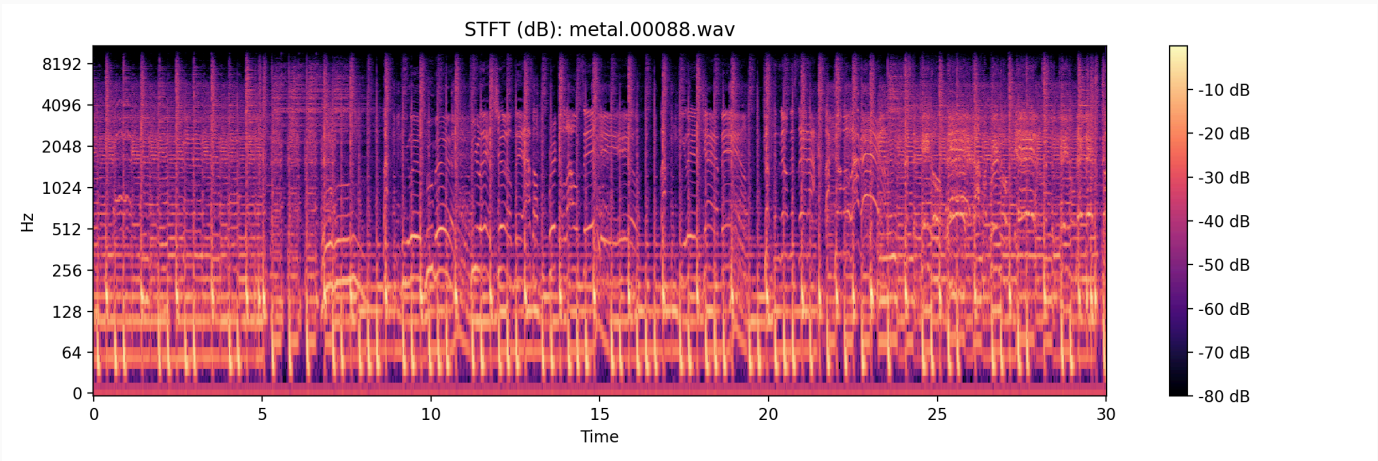
**What it captures.** - **Low-frequency structure** (bass lines, kick drum), - **high-frequency content** (hi-hats, distortion, brightness), - transient vs. harmonic texture (vertical vs. horizontal patterns).

**Figure 7–10.** STFT examples across genres.



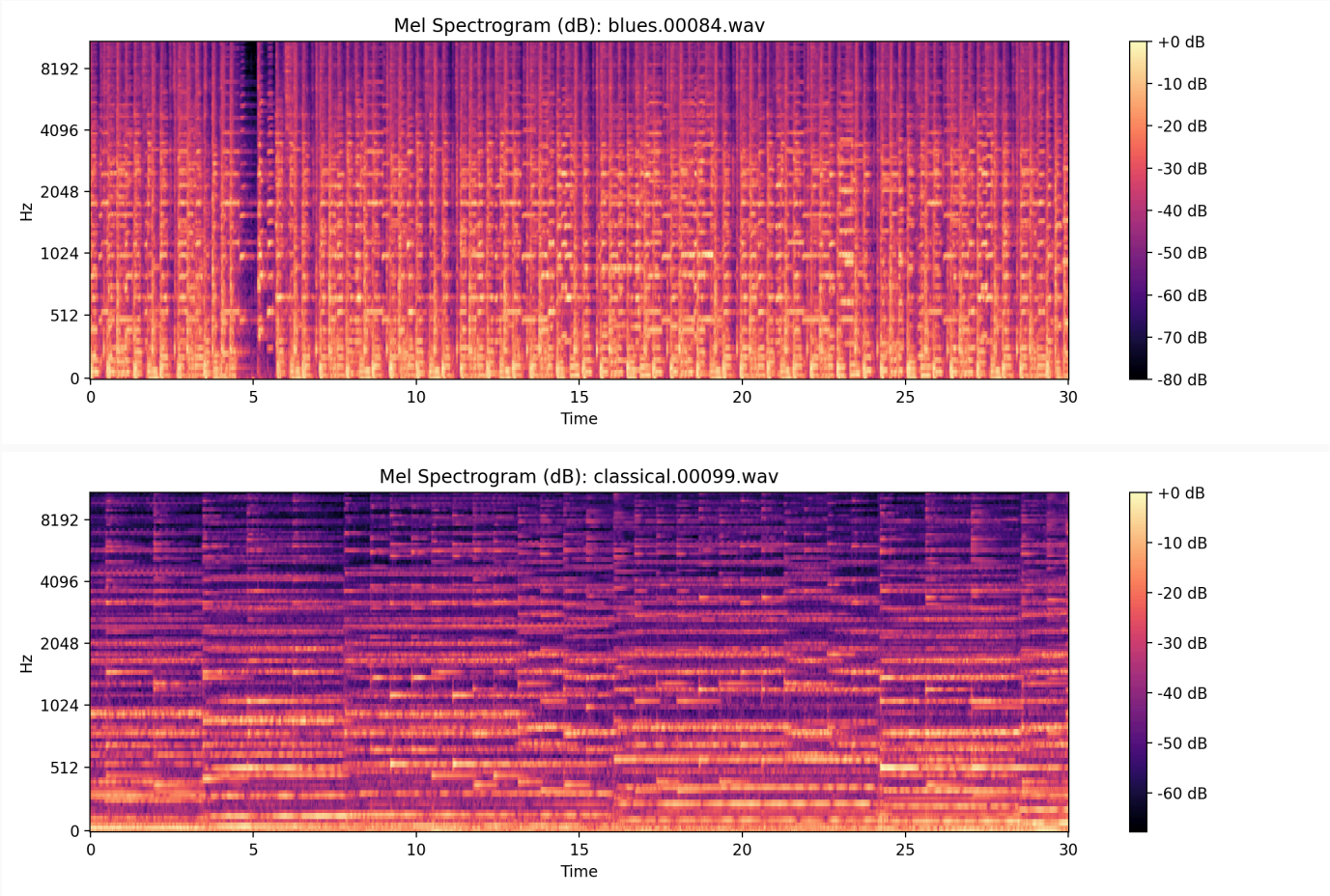

**Interpretation.** Metal tends to exhibit dense broadband energy (distorted guitars, cymbals), while classical often shows clearer harmonic bands and more separation between instruments. Hiphop can show strong low-frequency energy plus transient percussion patterns. These observations connect to engineered spectral features (centroid, bandwidth, rolloff) and to mel-spectrogram learning.

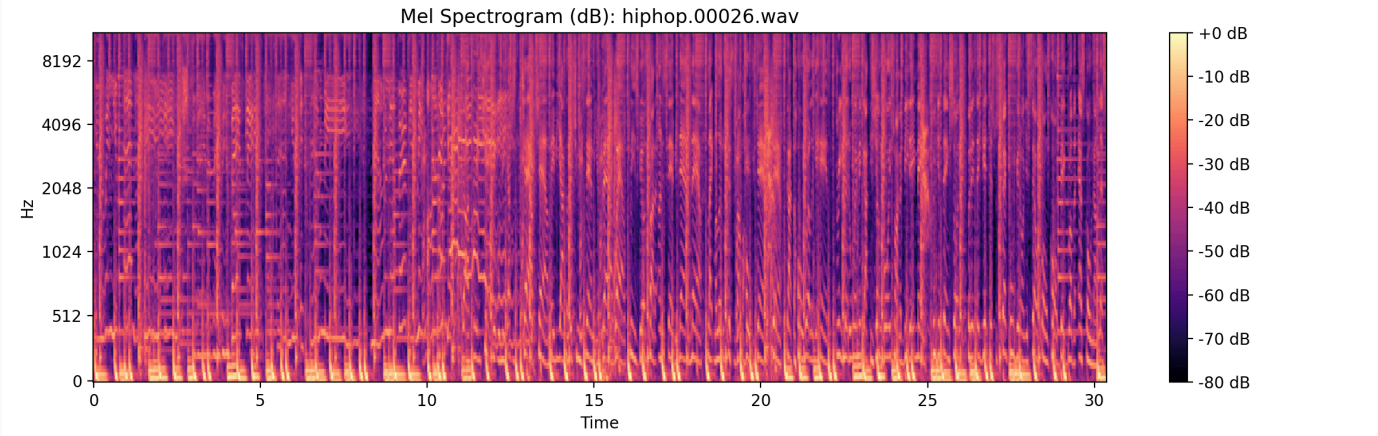
## 4.3 Mel spectrogram (dB) — the end-to-end representation

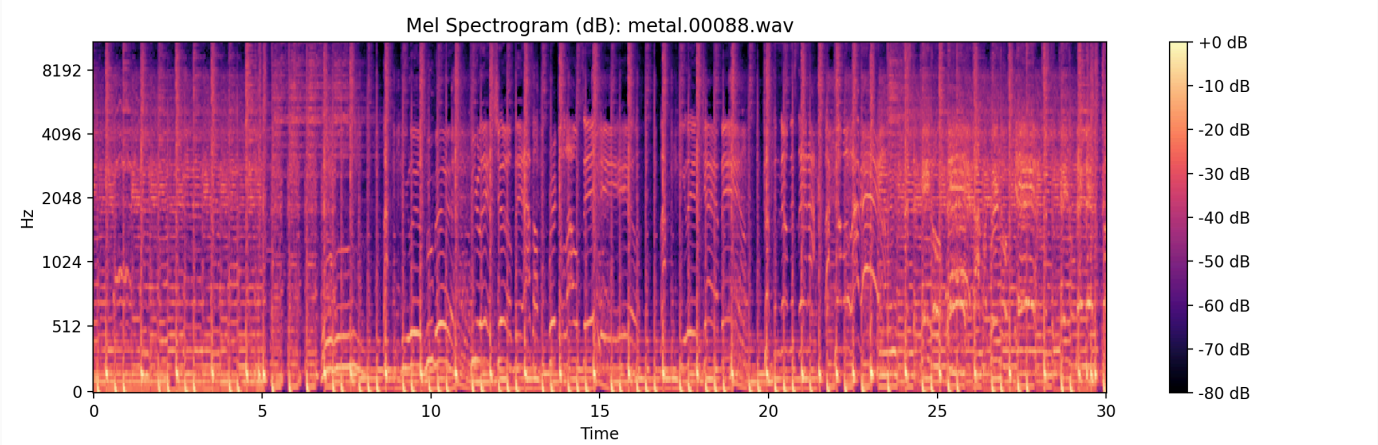
**Meaning.** The mel spectrogram projects a power spectrogram through a mel filterbank and then applies log compression: $m = 2595*{10}(1 + f/700), S{mel} = M S, S_{dB}=10*{10}(S{mel}+)$ In this pipeline: - `features.kind=mel_from_audio` extracts mel using `librosa.feature.melspectrogram` and `power_to_db`. - Configuration is in `configs/config.yaml`: - `sample_rate=22050`, `segment_seconds=3.0`, - `n_mels=128`, `n_fft=2048`, `hop_length=512`, - `fmin=20`, `fmax=8000`.

**Why useful.** Mel spectrograms preserve time–frequency structure and align frequency resolution with human perception, making them a strong input for CNN/LSTM.

**Figure 11–14.** Mel spectrogram examples.





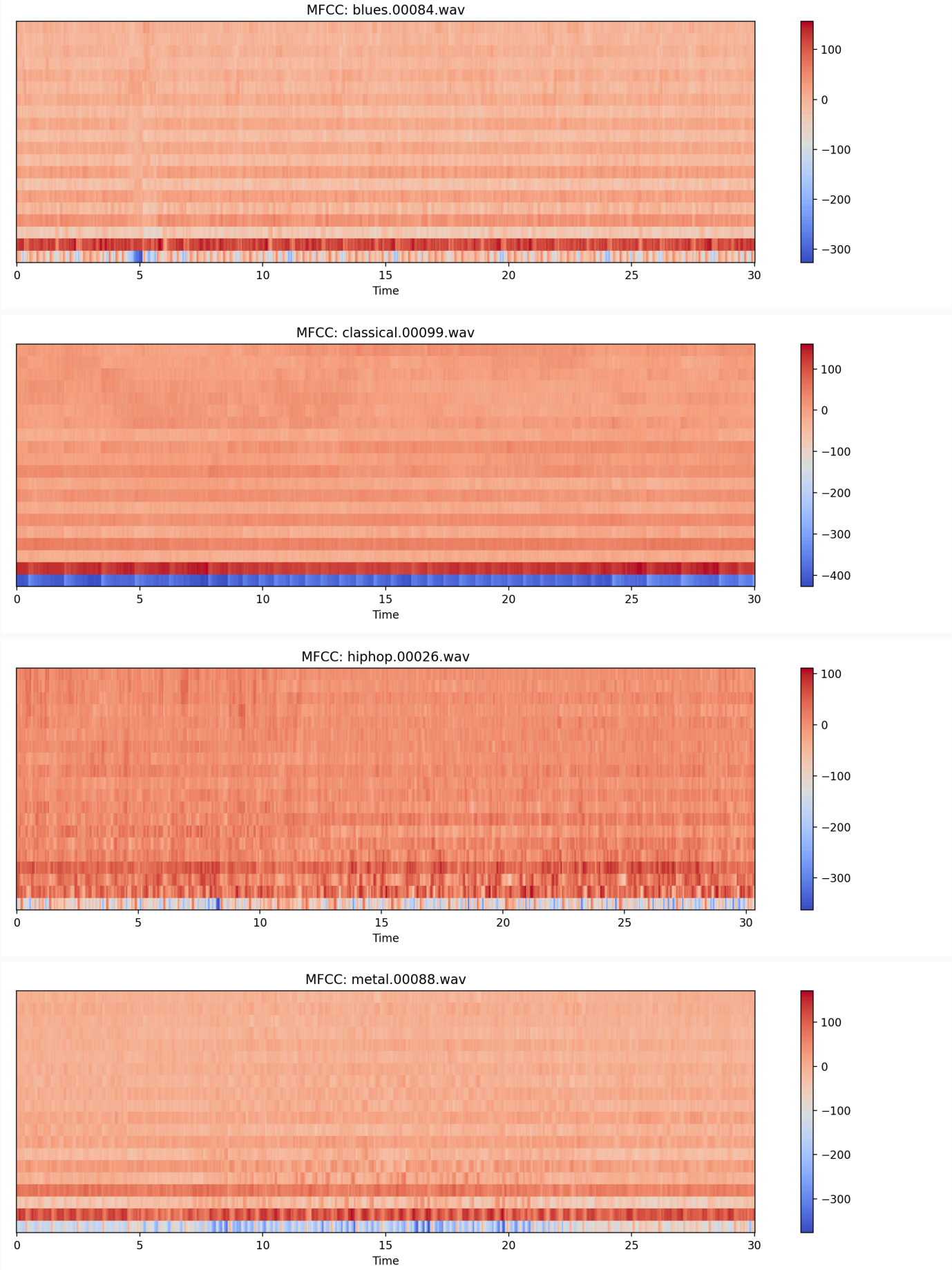




**Interpretation.** Relative energy distribution across mel bands (low vs. high) and temporal modulation patterns differentiate genres. For example, sustained harmonic textures appear as continuous horizontal structures; percussive events appear as vertical bursts.

## 4.4 MFCC (Mel-Frequency Cepstral Coefficients)

**Meaning.** MFCCs summarize the spectral envelope (timbre) via: 1) mel spectrogram $$ log, 2) Discrete Cosine Transform (DCT). MFCCs are widely used for speech/music timbre characterization and appear as the dominant engineered subfamily in `features_3_sec.csv` (`mfcc1..mfcc20_mean/var`).

**Figure 15–18.** MFCC heatmaps across genres.

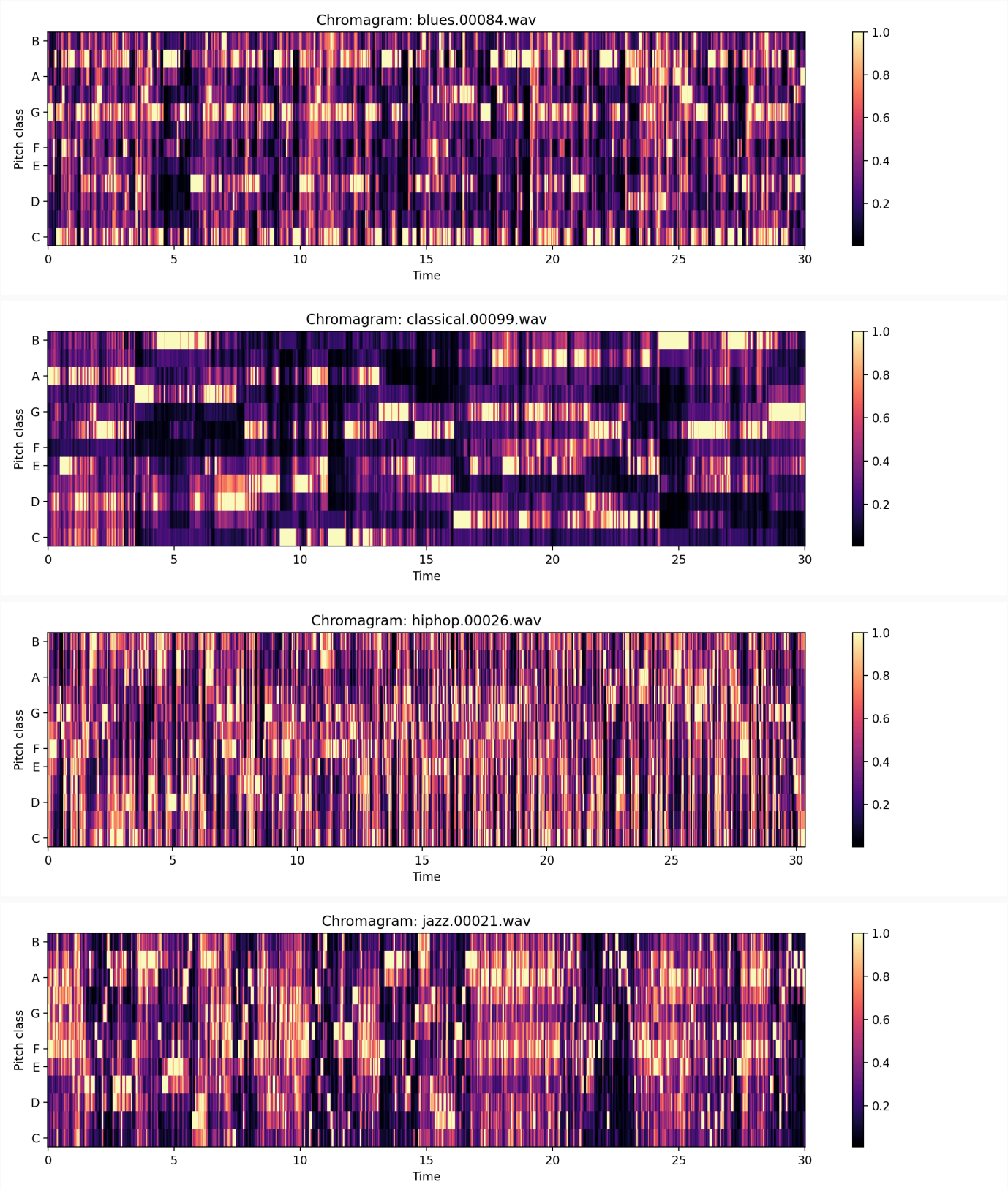



**Interpretation.** MFCCs compress spectral detail but preserve coarse timbre. Metal/rock timbres often exhibit dense high-frequency presence; classical timbres may show clearer harmonic structure and lower noise-like components. The tabular models heavily rely on MFCC means/variances (40 dimensions out of 58 inputs including `length`), which partly explains strong separability under leaky splitting.

## 4.5 Chromagram (pitch class energy)

**Meaning.** A chromagram maps energy to 12 pitch classes (C⋯B), discarding octave. It reflects harmonic progression and tonality.

**Figure 19–22.** Chromagrams across genres.




**Interpretation.** Genres with richer harmonic movement (jazz/classical) often display stronger and more structured chroma activation patterns; highly percussive segments can yield weaker or noisier chroma structure.

## 4.6 Spectral contrast (peak–valley structure across subbands)

**Meaning.** Spectral contrast measures energy differences between spectral peaks and valleys across frequency subbands. It is sensitive to instrumentation and timbral “texture”.

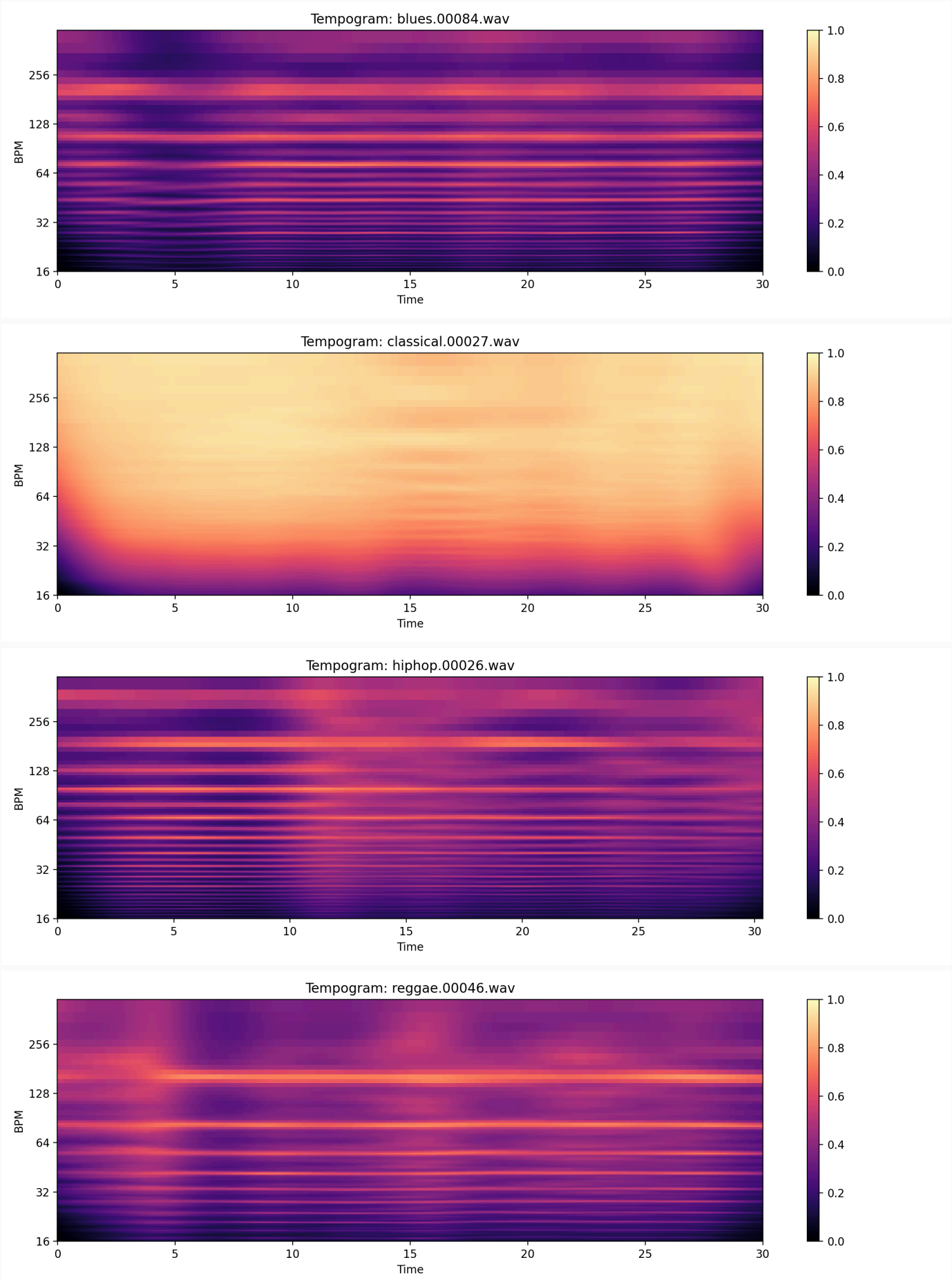
**Figure 23–26.** Spectral contrast examples.


**Interpretation.** Dense, compressed mixes often reduce contrast in some bands; sparse instrumentations can produce sharper peak–valley differences. This relates to genre-dependent production styles.

## 4.7 Tempogram (rhythmic periodicity)

**Meaning.** The tempogram is computed from onset strength over time and captures rhythmic periodicities (tempo-related structure).

**Figure 27–30.** Tempograms across genres.

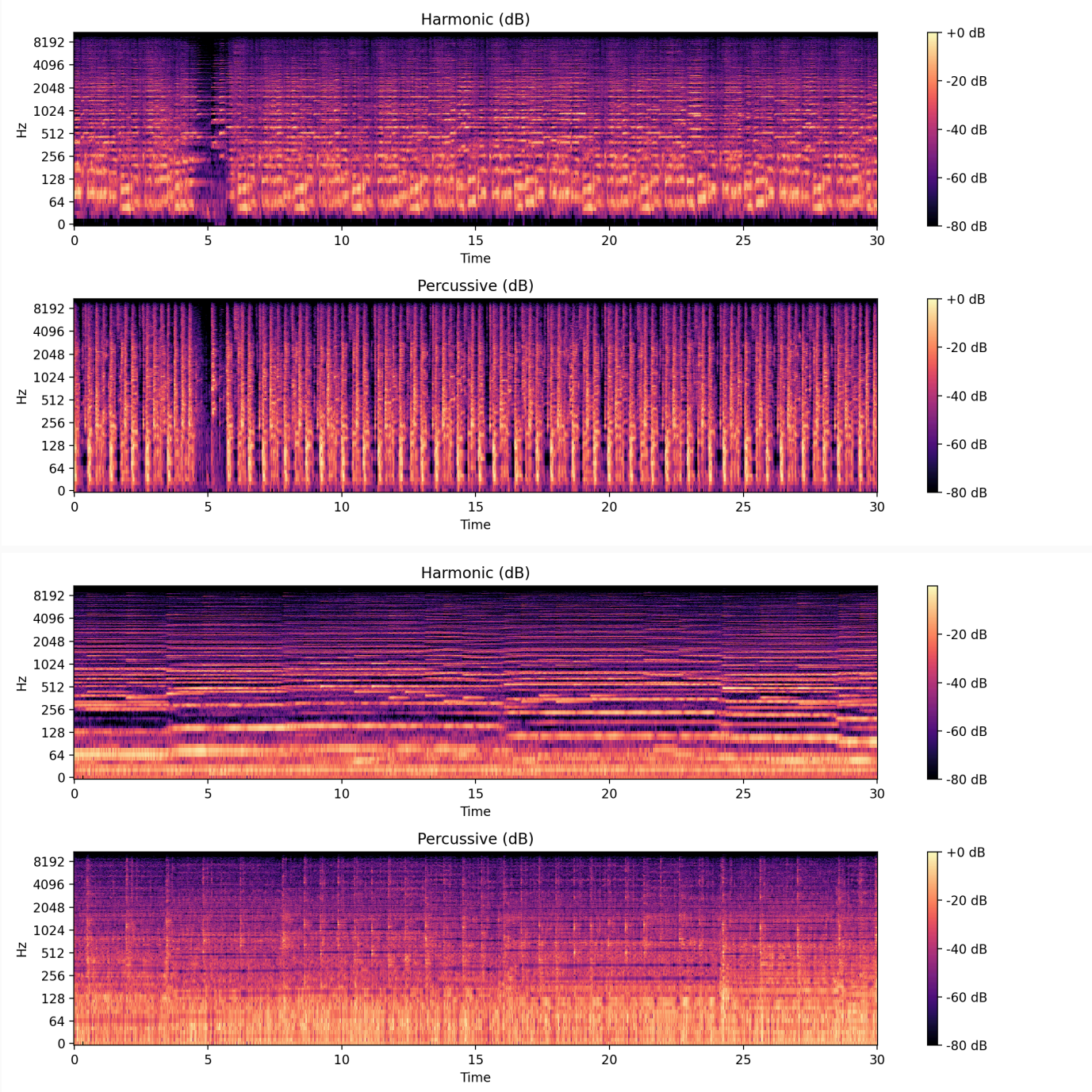


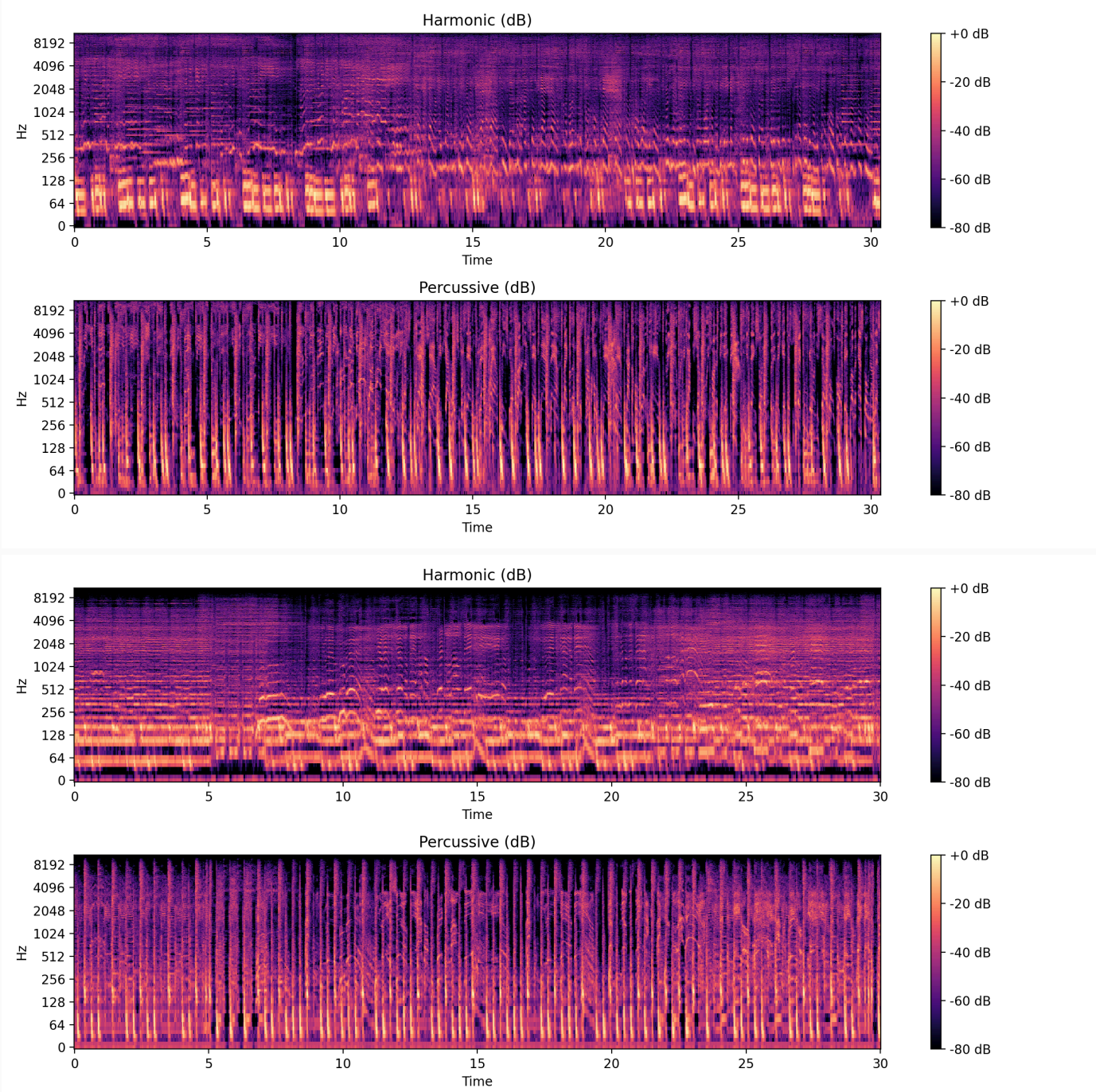

**Interpretation.** Disco/hiphop typically show stronger periodic rhythmic patterns over short windows, whereas classical can be more variable. These properties motivate inclusion of `tempo` in the tabular feature set.

## 4.8 HPSS (harmonic/percussive source separation)

**Meaning.** HPSS decomposes the STFT into harmonic (sustained) and percussive (transient) components. The tabular feature CSV includes `harmony_*` and `perceptr_*` statistics, and the project provides HPSS visualizations.

**Figure 31–34.** HPSS visualizations across genres.






**Interpretation.** Classical/jazz segments often contain strong harmonic structure; hiphop/disco frequently show pronounced percussive energy and transient patterns. The harmonic/percussive balance is a meaningful discriminant for genre texture.

## 4.9 Tabular feature space diagnostics (correlation + embeddings)

The tabular feature cache is standardized (z-score) on the training split and then visualized via correlation heatmaps and 2D embeddings.

**Figure 35.** Feature correlation heatmap (subset of first 40 features).

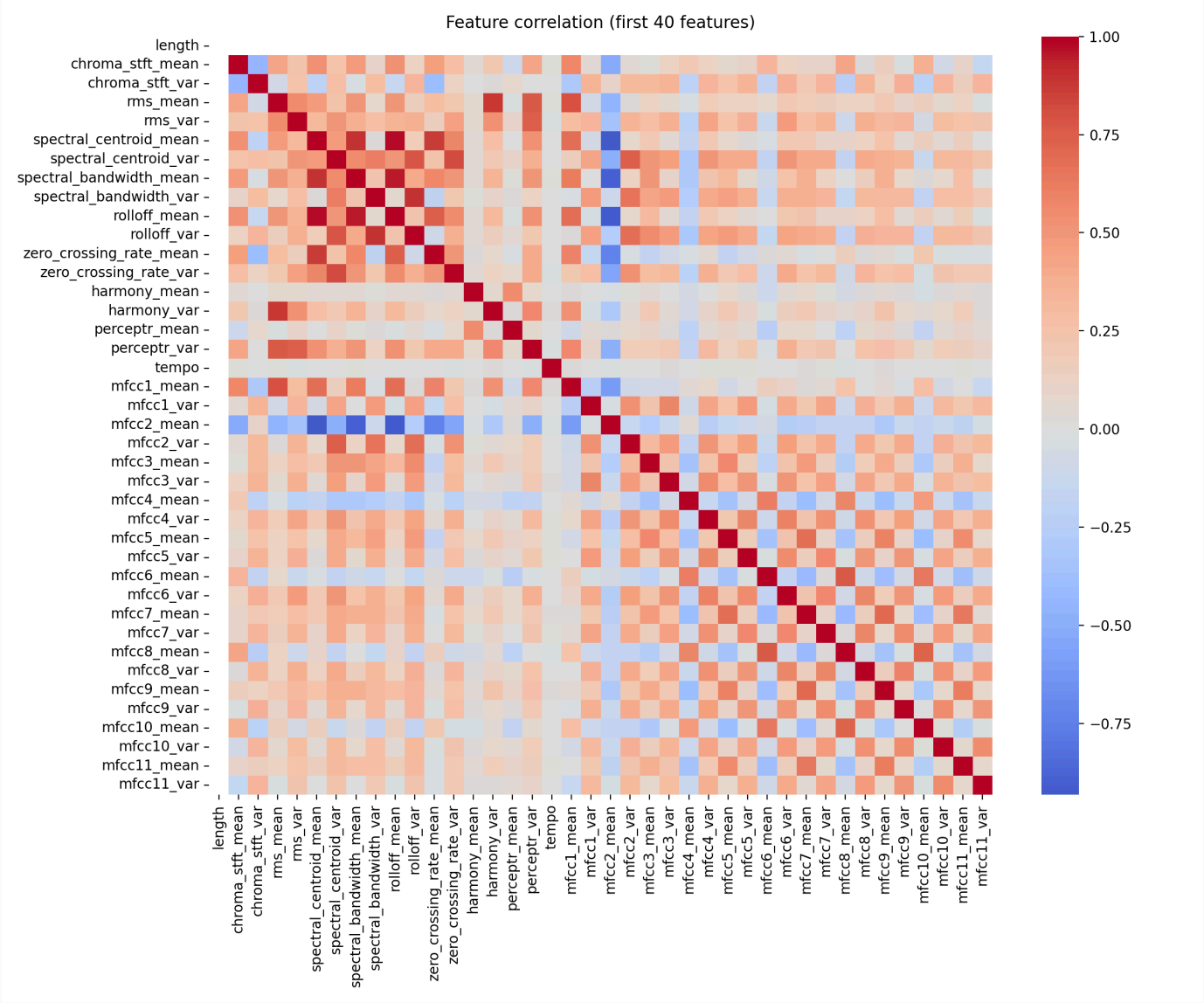
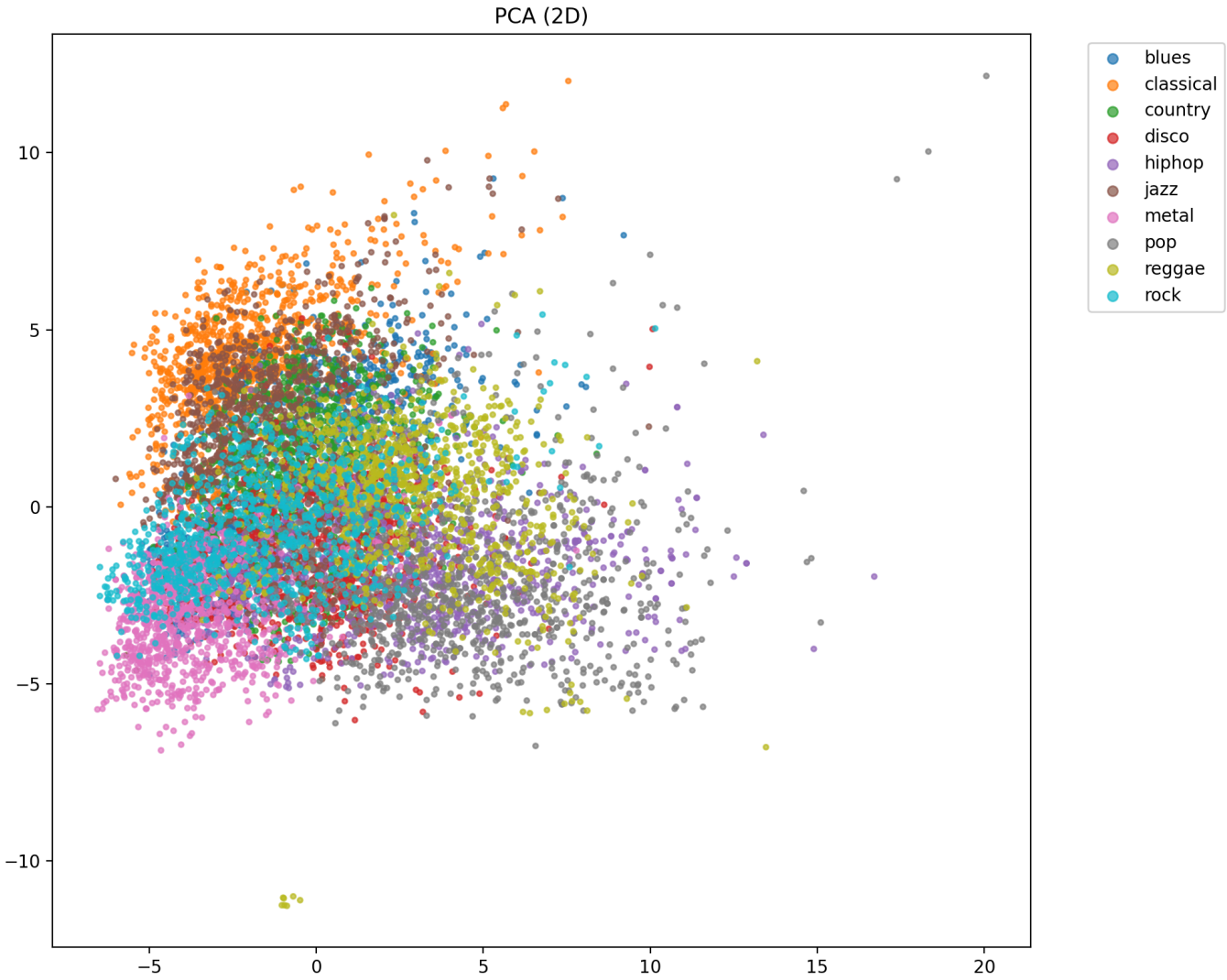
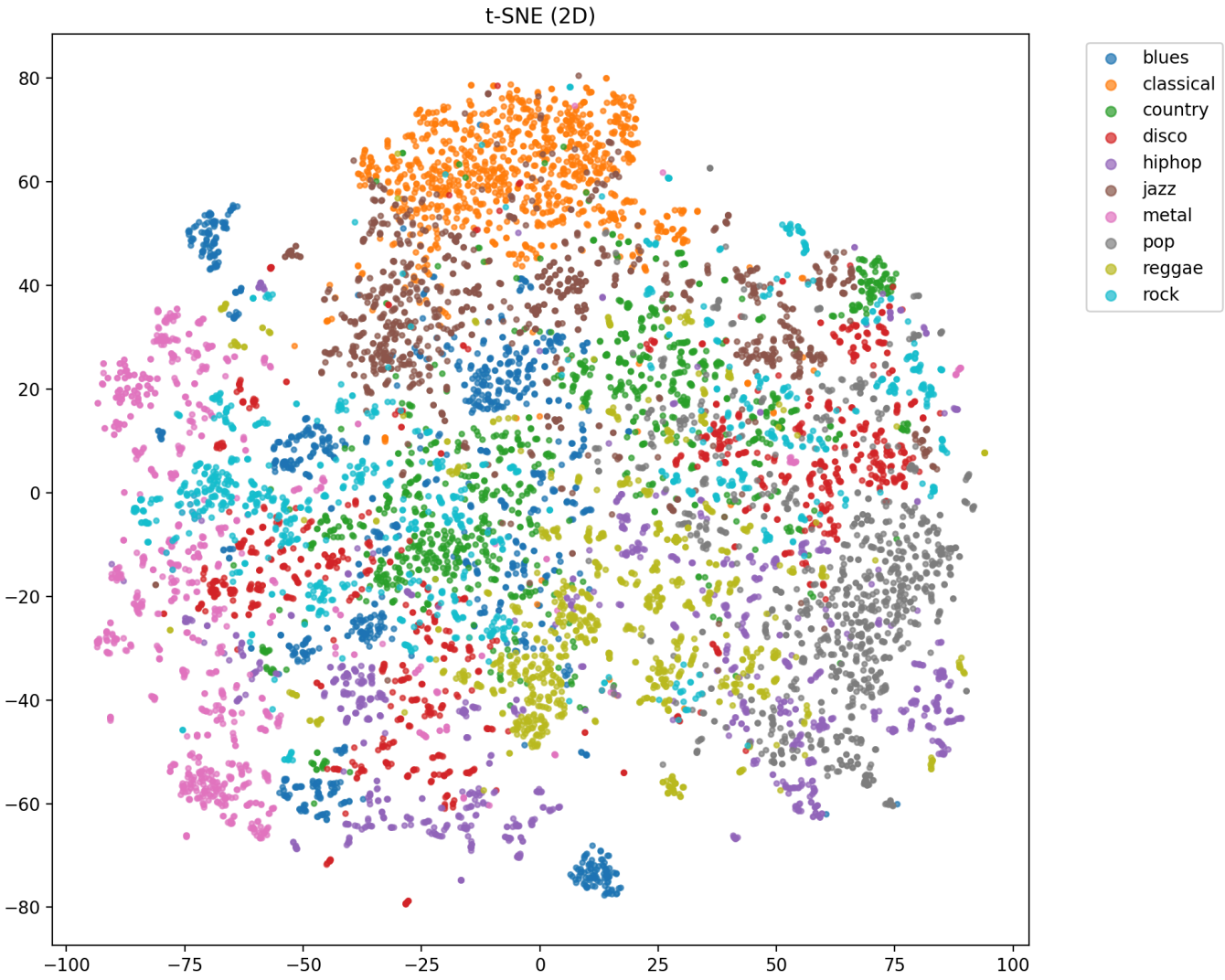
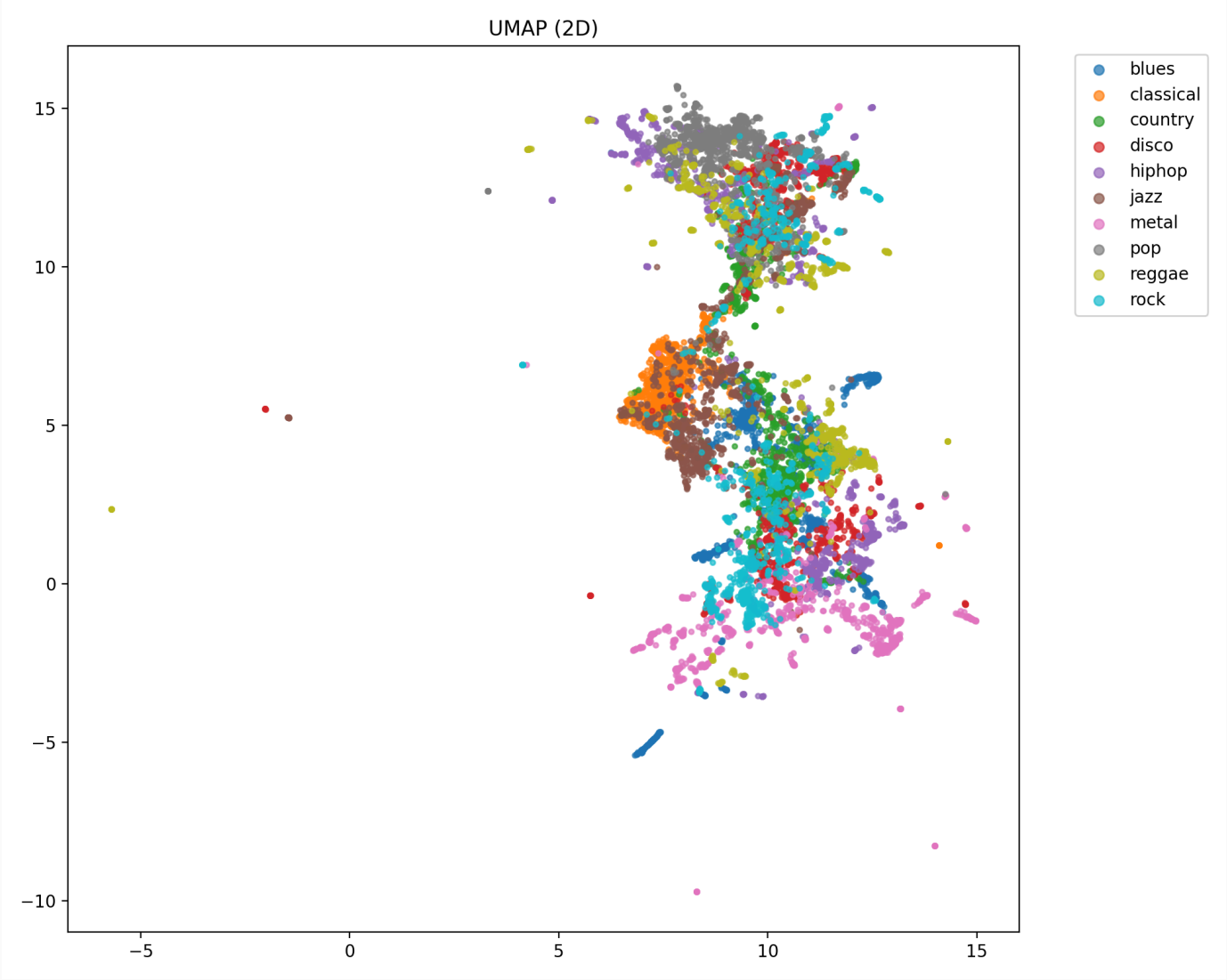



Figure 35: Feature correlation heatmap for tabular inputs.

**Figure 36–38.** Embeddings of tabular features.


PCA (2D)
blues
classical
country
disco
hiphop
jazz
metal
pop
reggae
rock
10
5
0
−5
−10
−5
0
5
10
15
20


t-SNE (2D)
blues
classical
country
disco
hiphop
jazz
metal
pop
reggae
rock
80
60
40
20
0
−20
−40
−60
−80
−100
−75
−50
−25
0
25
50
75
100




**Interpretation.** Clear clustering in these embeddings can indicate separability of classes in the engineered feature space. However, because the tabular split is not track-disjoint, apparent separation may partially reflect track-specific signatures leaking across splits.

---

# 5. Model Architecture Analysis

The repository includes a “model zoo” spanning classical ML (scikit-learn) and neural models (PyTorch). Model definitions and configuration live in: - `src/models/` (implementations) - `configs/model.yaml` (hyperparameters)

## 5.1 Classical ML baselines (tabular, scikit-learn)

Implemented via `src/models/sklearn_models.py` with supported names: `knn, svm_rbf, linear_svm, logreg, naive_bayes, lda, qda, random_forest, extra_trees, ada_boost, gbdt, xgboost, lightgbm` .

**Rationale.** These methods are strong and efficient for low-dimensional tabular data. Ensemble tree methods (ExtraTrees, RandomForest, GBDT, XGBoost, LightGBM) can capture non-linear feature interactions without extensive preprocessing beyond standardization.

**Notable implementation details.** - `svm_rbf` uses `SVC(probability=True)` to enable ROC/PR curves (computationally heavier but informative). - `linear_svm` uses `LinearSVC` , which does **not** produce probability estimates; therefore ROC-AUC and PR curves may be unavailable for that run.

## 5.2 MLP for tabular features (PyTorch)

Defined in `src/models/model.py::MLP` : - Input: 58D vector - Architecture: `Dropout -> (Linear + ReLU + Dropout) × 5 -> Linear(n_classes)` - Hidden sizes: `[512, 256, 128, 64, 32]` , dropout `0.2` (from `configs/model.yaml` )

**Strengths/weaknesses.** - Flexible non-linear function approximator. - Requires careful regularization and sufficient data; can overfit under leakage or small sample regimes.

## 5.3 CNN on mel spectrogram (PyTorch)

Defined in `src/models/cnn.py::CNNMel` : - Input: mel tensor shape `(B, 1, n_mels, time)` - Blocks: `Conv2d -> BatchNorm -> ReLU -> MaxPool2d -> Dropout` (repeated 3 times) - Final: `AdaptiveAvgPool2d((1,1)) -> Linear -> logits` - Channels: `[32, 64, 128]` , dropout `0.3`

**Rationale.** Convolution captures local time–frequency patterns such as rhythmic transients, harmonic stacks, and spectral envelopes.

## 5.4 LSTM on mel sequences (PyTorch)

Defined in `src/models/lstm.py::LSTMMel` : - Input: sequence shape `(B, time, n_mels)` (mel image reshaped to a time series) - 2-layer bidirectional LSTM with hidden size 128 and dropout 0.2 - Head: `LayerNorm -> Dropout -> Linear(n_classes)`

**Rationale.** LSTM models temporal dependencies across frames (e.g., evolving rhythm/harmony).

**Trade-off.** LSTM can be more sensitive to optimization and may underperform a CNN for spectrogram-like inputs when spatial locality in frequency is important.

---

# 6. Training Configuration

Configuration is centralized in `configs/config.yaml` and `configs/train.yaml` .

## 6.1 Global settings

- Seed: `42`
- Device: auto-detect ( `cuda > mps > cpu` ) with overrides supported ( `--device` ).
- DataLoader workers: `2`

## 6.2 Tabular training (torch MLP)

- Epochs: `120`
- Batch size: `256` (train), `512` (eval)
- Optimizer: Adam ( `configs/train.yaml` )
- Learning rate: `1.46e-4`
- Weight decay: `1.0e-4`
- Scheduler: ReduceLROnPlateau with factor `0.5` , patience `5` , min LR `1e-6`
- Early stopping: enabled, patience `15` , monitor `val_loss` , mode `min`

## 6.3 Mel feature extraction (end-to-end track)

- Sampling: `22050 Hz` , mono
- Segment length: `3.0 s`
- STFT: `n_fft=2048` , `hop_length=512`
- Mel bands: `128` , `fmin=20` , `fmax=8000`

## 6.4 Data augmentation strategies (configured in `data.augmentation` )

Three challenges motivated the following augmentations, all implemented in `src/data/augmentation.py` :

### Genre Overlap → Mixup ( `data.augmentation.mixup` )

Mixup interpolates training sample pairs to expose models to synthetic boundary examples: $$x_{} = x_i + (1-) x_j, y_{} = y_i + (1-) y_j, (, )$$ This regularises the model to predict smooth transitions between genre distributions, rather than committing to sharp decision boundaries. Effective for genres sharing instrumentation (rock/metal, blues/country).

| Parameter | Default | Notes |
|---|---|---|
| `enabled` | `false` | Enable for MLP/CNN/LSTM training |
| `alpha` | `0.2` | Controls interpolation strength; higher = more mixing |
| `prob` | `1.0` | Fraction of batches to apply |

### Domain Shift → SpecAugment ( `data.augmentation.spec_augment` )

SpecAugment (Park et al., 2019) masks contiguous frequency bands and time frames of the mel spectrogram: - **Frequency masking**: zeroes out $f$ consecutive mel bins, simulating microphone roll-off or band-limited environments. - **Time masking**: zeroes out $t$ consecutive time frames, simulating recording dropouts or noise bursts.

This forces the model to rely on the remaining intact signal rather than memorising specific frequency/time signatures of the training domain.

| Parameter | Default | Notes |
|---|---|---|
| `enabled` | `false` | Only applicable for `mel_from_audio` |
| `freq_mask_param` | `20` | Max mel bins to mask |
| `time_mask_param` | `20` | Max time frames to mask |
| `n_freq_masks` | `2` | Number of frequency masks |
| `n_time_masks` | `2` | Number of time masks |

### Data Leakage → Fixed at split level (no augmentation needed)

Track-level leakage is **structurally eliminated** by the group-level split ( `make_group_splits_from_filenames` ). No augmentation can substitute for a correct split.

---

# 7. Experimental Results

All experimental artifacts referenced here are stored under `outputs/` in per-run directories with: - logs: `outputs/<run>/logs/*.log` - figures: `outputs/<run>/figures/*.png` - reports: `outputs/<run>/reports/*`

## 7.1 Cross-model summary (from evaluation reports)

The table below aggregates metrics from `outputs/*/reports/evaluation_report.md` and approximate training time from the last session in `outputs/*/logs/train.log` .

| Run | Accuracy | Macro-F1 | ROC-AUC (OvR, macro) | Train time (s) |
|---|---|---|---|---|
| tabular_lightgbm | 0.9129 | 0.9127 | 0.9955 | 13 |
| tabular_svm_rbf | 0.9069 | 0.9068 | 0.9946 | 7 |

| Run | Accuracy | Macro-F1 | ROC-AUC (OvR, macro) | Train time (s) |
|---|---|---|---|---|
| tabular_xgboost | 0.9044 | 0.9042 | 0.9946 | 16 |
| tabular_extra_trees | 0.8904 | 0.8900 | 0.9902 | 3 |
| tabular_knn | 0.8744 | 0.8740 | 0.9720 | 6 |
| tabular_random_forest | 0.8694 | 0.8687 | 0.9887 | 3 |
| tabular_gbdt | 0.8544 | 0.8549 | 0.9886 | 814 |
| tabular_mlp | 0.8078 | 0.8064 | 0.9777 | 2092 |
| tabular_qda | 0.7487 | 0.7486 | 0.9633 | 0 |
| tabular_logreg | 0.7152 | 0.7129 | 0.9565 | 4 |
| tabular_linear_svm | 0.6977 | 0.6919 | — | 1 |
| tabular_lda | 0.6627 | 0.6611 | 0.9403 | 0 |
| mel_cnn | 0.6310 | 0.6211 | 0.9211 | 1602 |
| mel_lstm | 0.5195 | 0.5168 | 0.8782 | 1185 |
| tabular_naive_bayes | 0.4990 | 0.4795 | 0.8807 | 0 |
| tabular_ada_boost | 0.4299 | 0.4155 | 0.8409 | 26 |

**Important note (validity).** The tabular runs above are evaluated on a segment split with **track leakage** (Section 3.4.1). The mel runs are evaluated on a **track-disjoint** split and should be regarded as a more faithful estimate of generalization to unseen tracks.

## 7.2 End-to-end mel models: CNN vs. LSTM

### 7.2.1 Training curves

**Figure 39.** CNN training curves ( `outputs/mel_cnn/figures/training_curves.png` ).

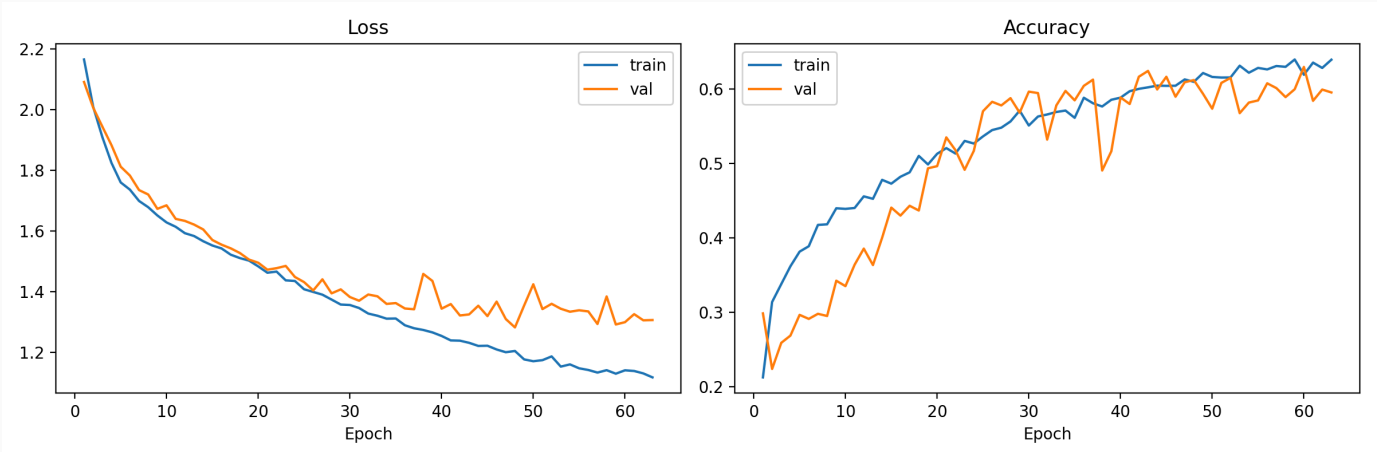



Figure 39: Training curves for mel CNN.

**Figure 40.** LSTM training curves ( `outputs/mel_lstm/figures/training_curves.png` ).

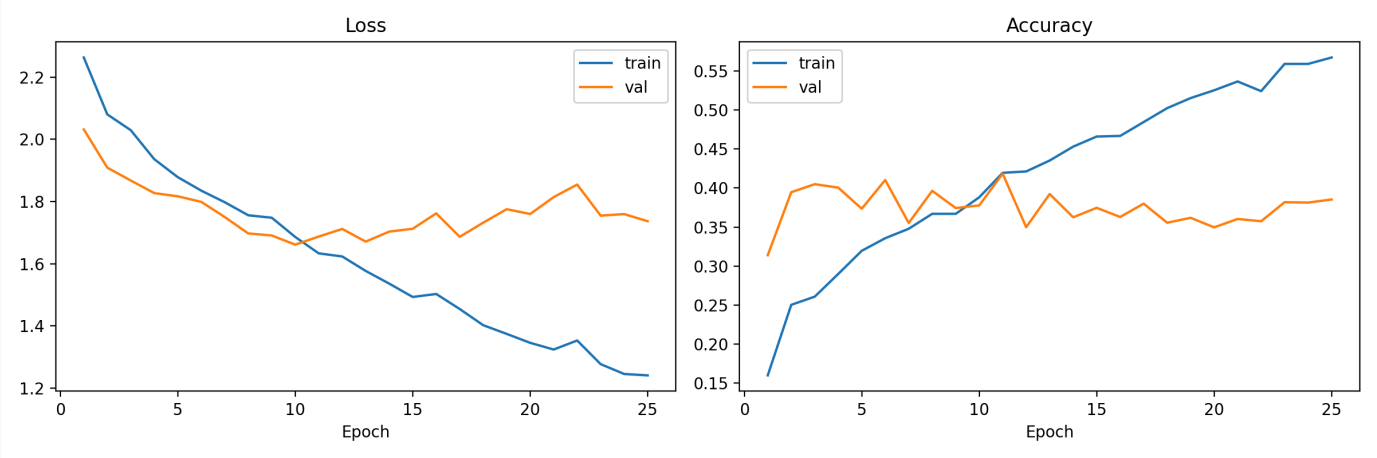
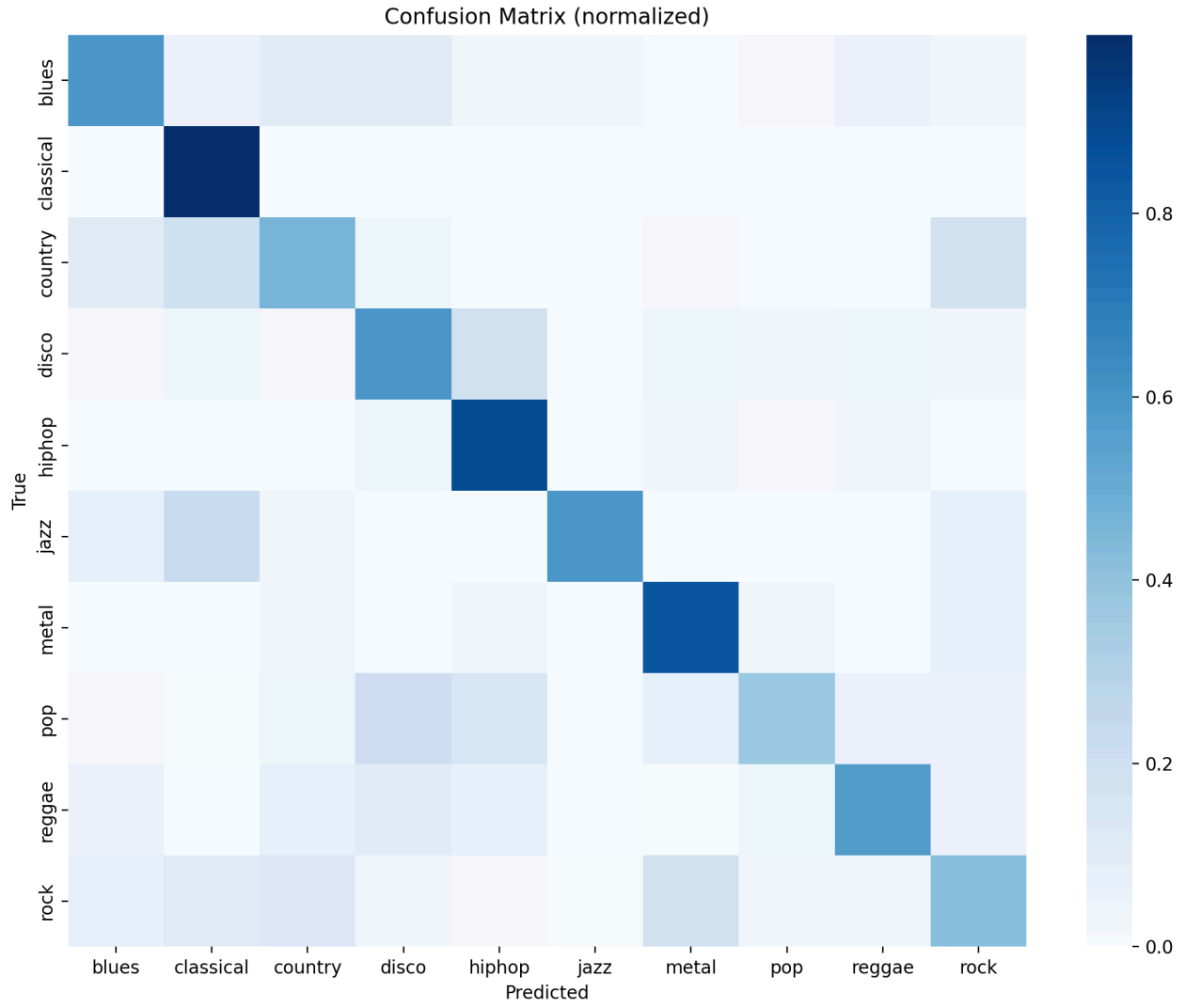
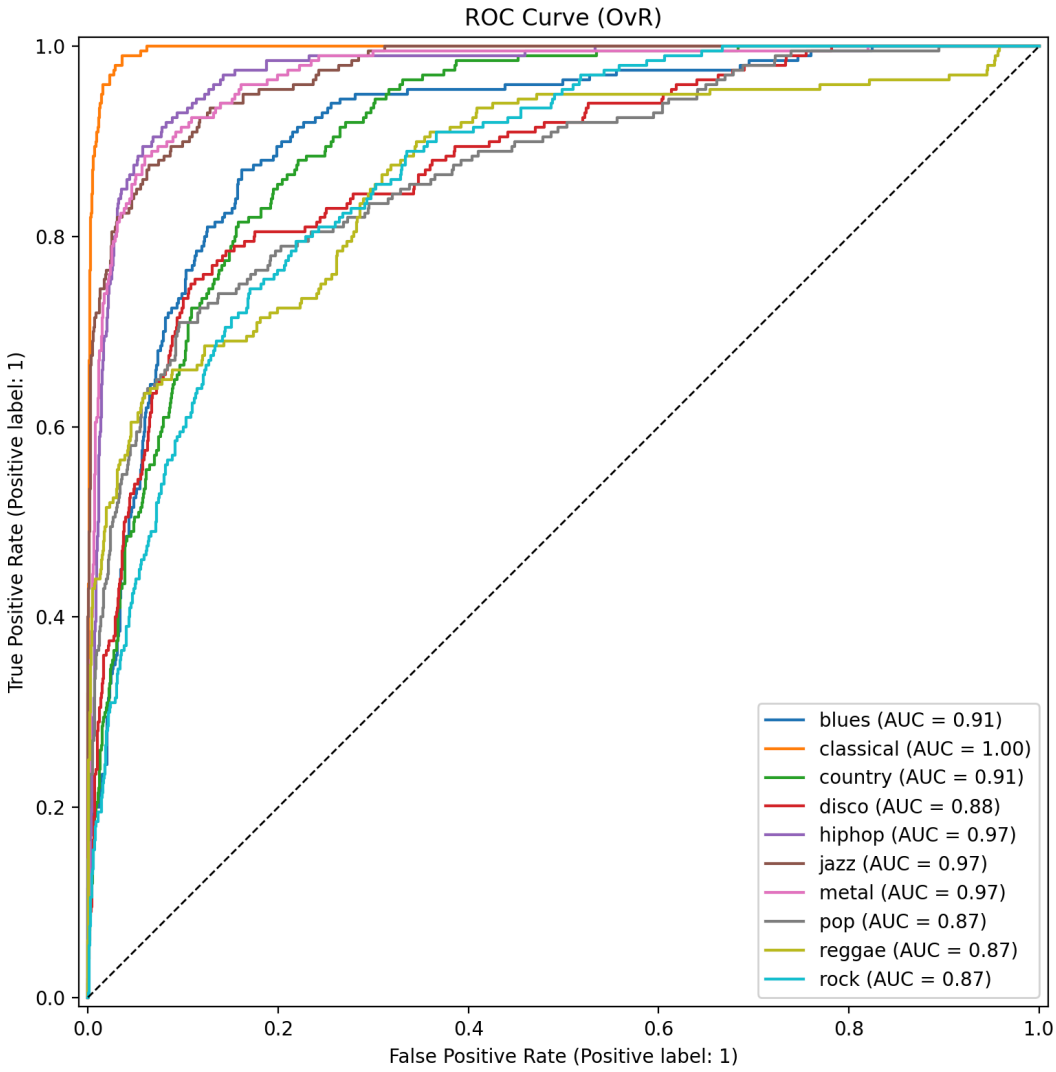



Figure 40: Training curves for mel LSTM.

### 7.2.2 Confusion matrices and ranking curves

**Figure 41–43.** CNN evaluation plots.


Confusion Matrix (normalized)
True
Predicted
blues
classical
country
disco
hiphop
jazz
metal
pop
reggae
rock
0.0
0.2
0.4
0.6
0.8


ROC Curve (OvR)
True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
0.0
0.2
0.4
0.6
0.8
1.0
blues (AUC = 0.91)
classical (AUC = 1.00)
country (AUC = 0.91)
disco (AUC = 0.88)
hiphop (AUC = 0.97)
jazz (AUC = 0.97)
metal (AUC = 0.97)
pop (AUC = 0.87)
reggae (AUC = 0.87)
rock (AUC = 0.87)

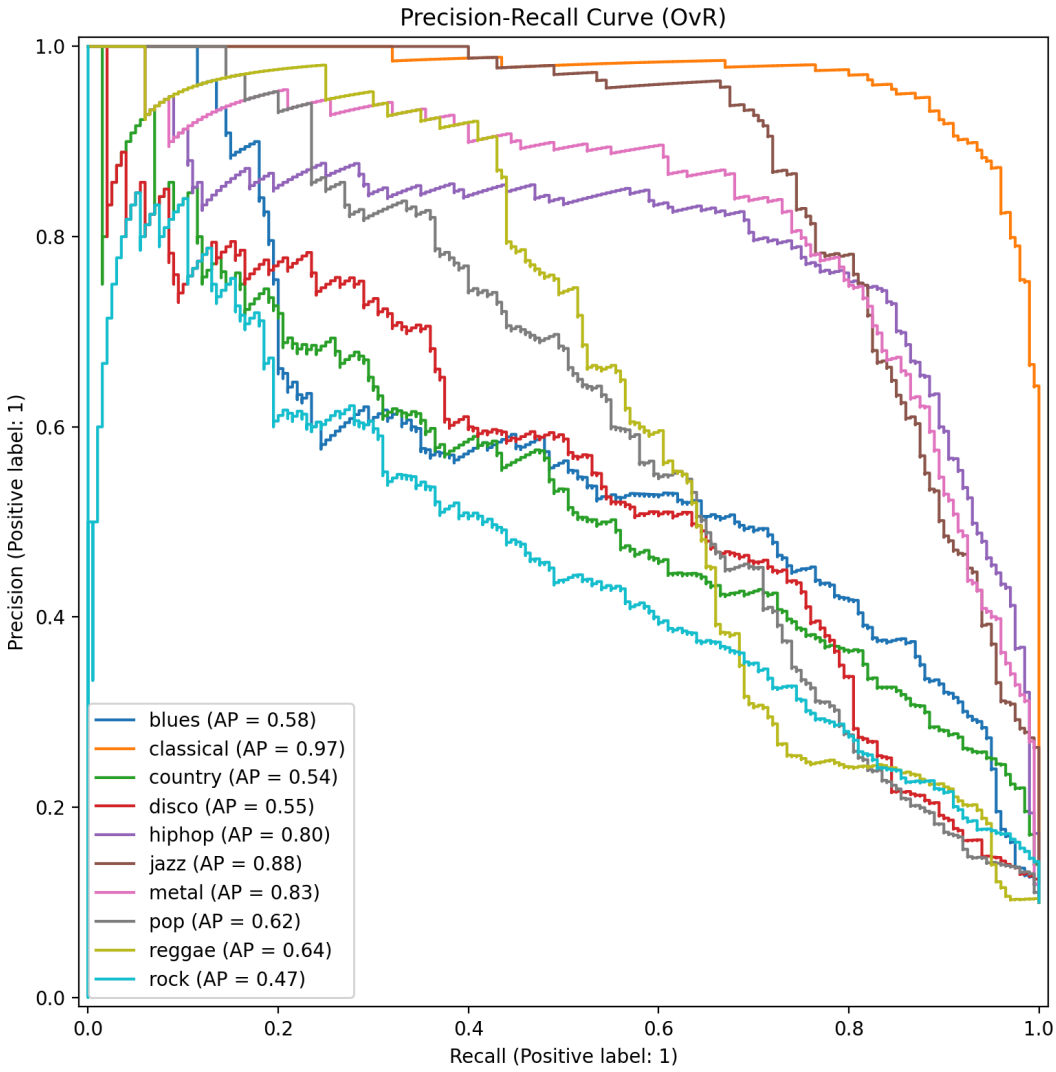
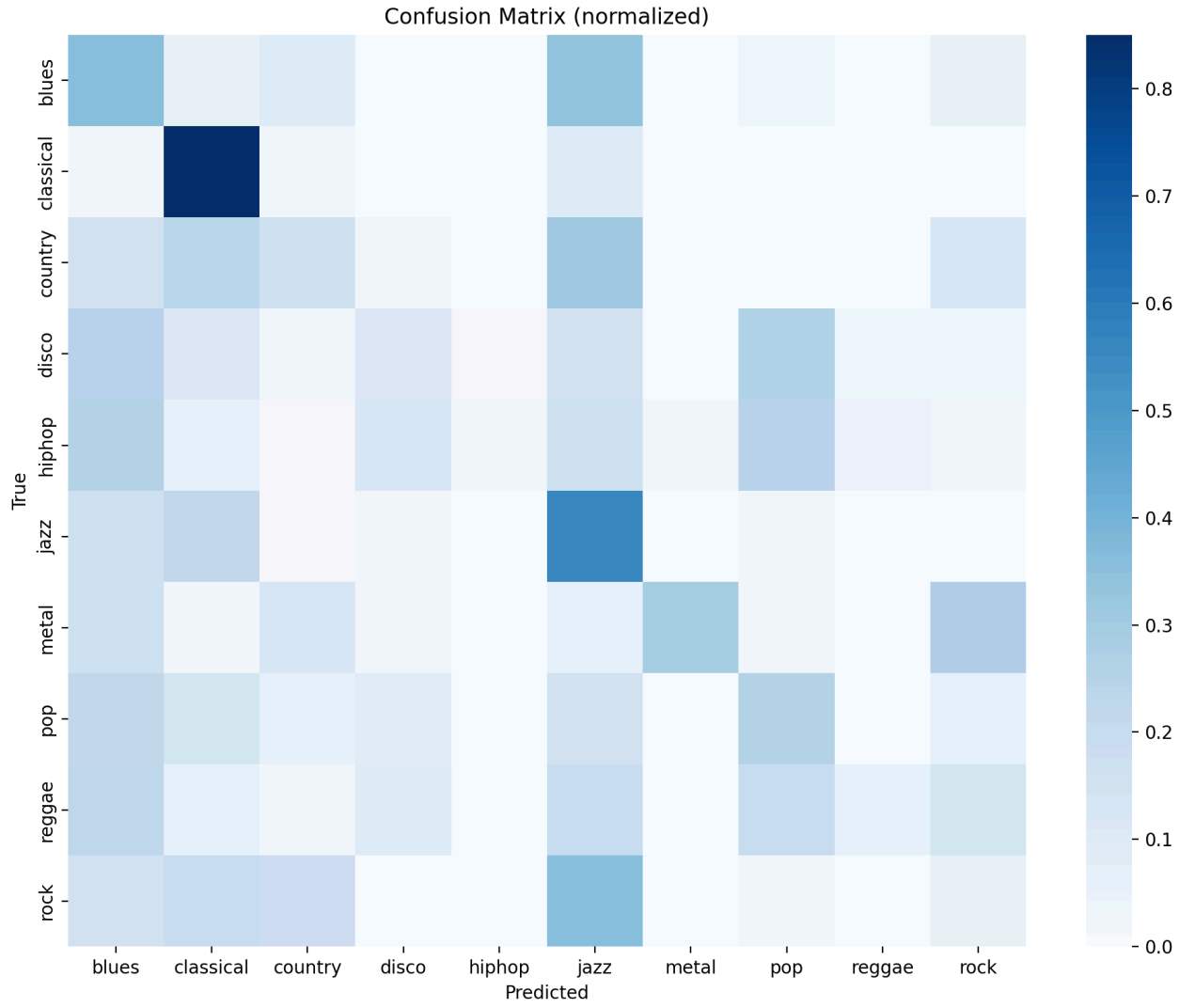
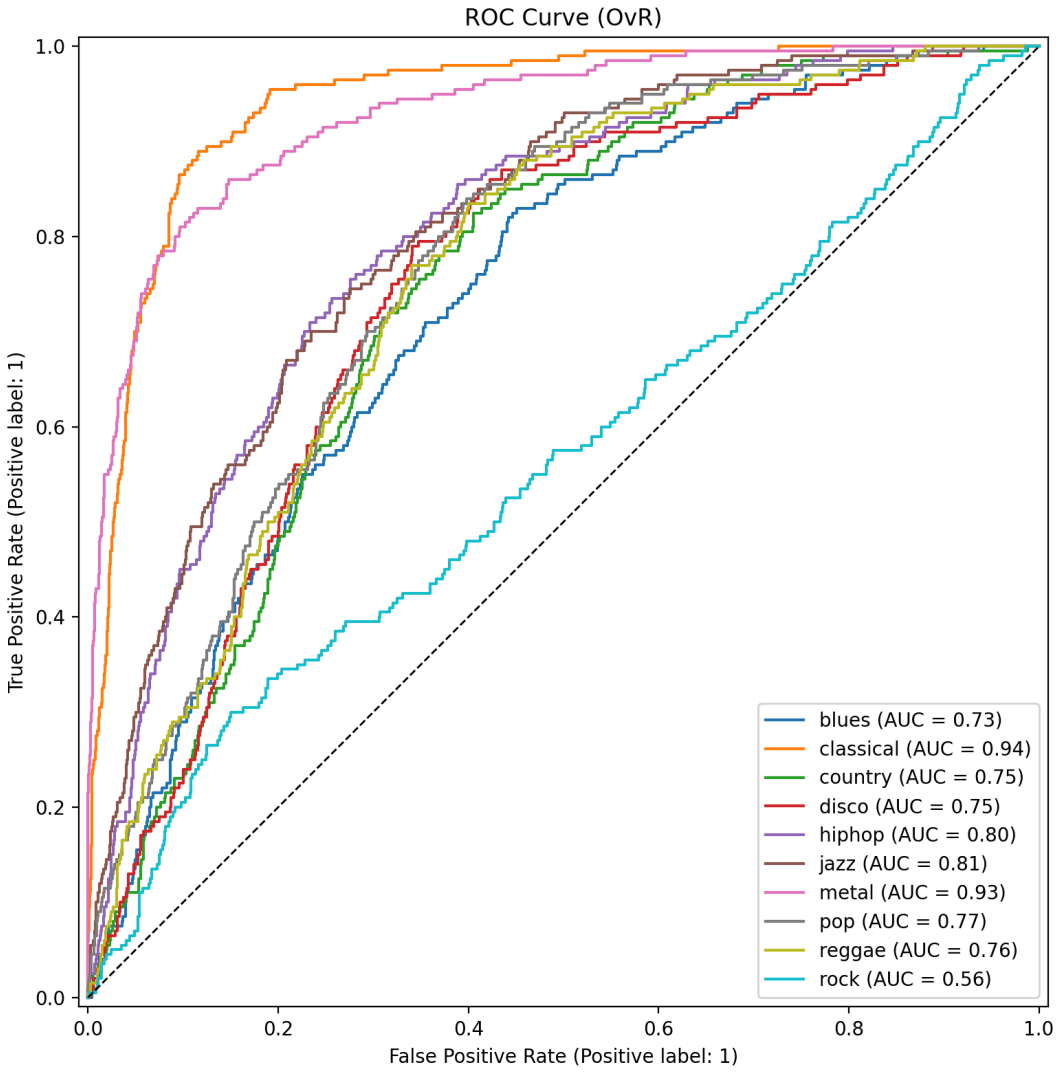
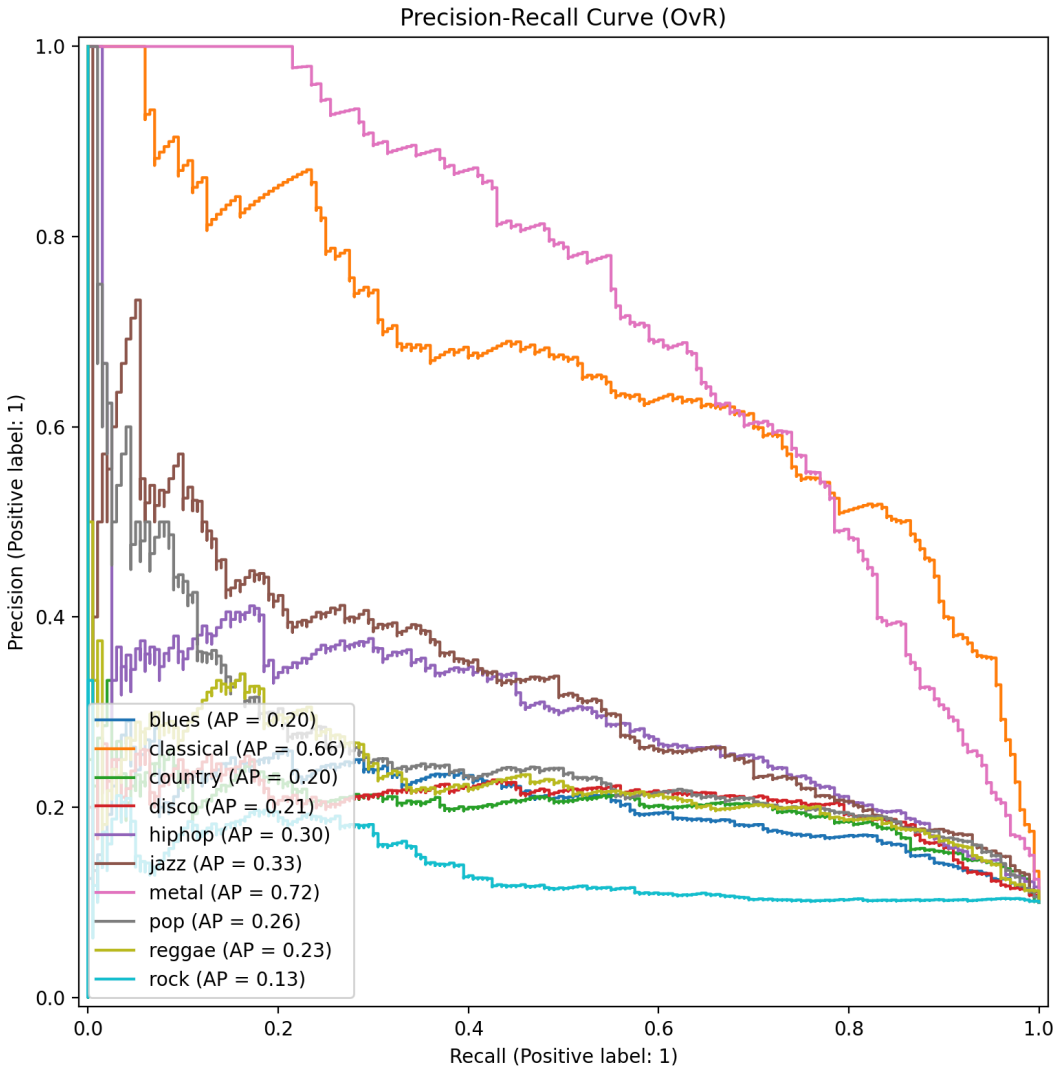



**Figure 44–46.** LSTM evaluation plots.


Confusion Matrix (normalized)
True
Predicted
blues
classical
country
disco
hiphop
jazz
metal
pop
reggae
rock
0.8
0.7
0.6
0.5
0.4
0.3
0.2
0.1
0.0


ROC Curve (OvR)
True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
0.0
0.2
0.4
0.6
0.8
1.0
blues (AUC = 0.73)
classical (AUC = 0.94)
country (AUC = 0.75)
disco (AUC = 0.75)
hiphop (AUC = 0.80)
jazz (AUC = 0.81)
metal (AUC = 0.93)
pop (AUC = 0.77)
reggae (AUC = 0.76)
rock (AUC = 0.56)




### 7.2.3 Class-wise performance highlights (from evaluation reports)

For `mel_cnn`, the lowest F1 classes are: - rock (F1 ≈ 0.4377), country (≈ 0.4741), pop (≈ 0.4935).

For `mel_lstm`, overall accuracy is lower (0.5195), consistent with the architectural trade-off: treating mel as a 1D time sequence can discard useful local frequency neighborhoods that CNNs exploit.

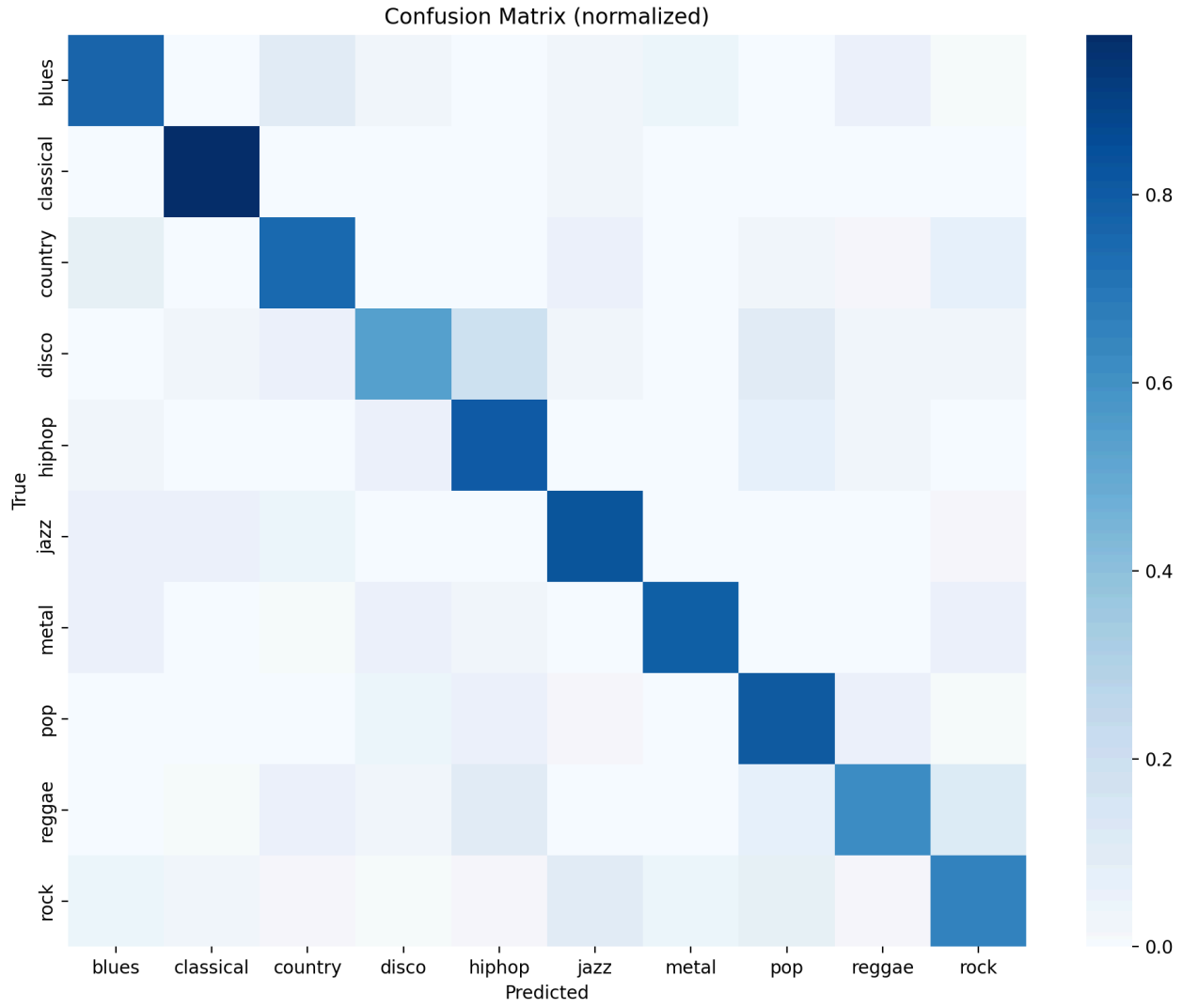
**Observed error patterns (from `outputs/mel_cnn/reports/misclassified.csv`, first 200 errors).** The most frequent confusions include: - country → classical / rock, - blues → country / disco, indicating that short 3-second segments can be ambiguous and that some genre cues require longer temporal context.

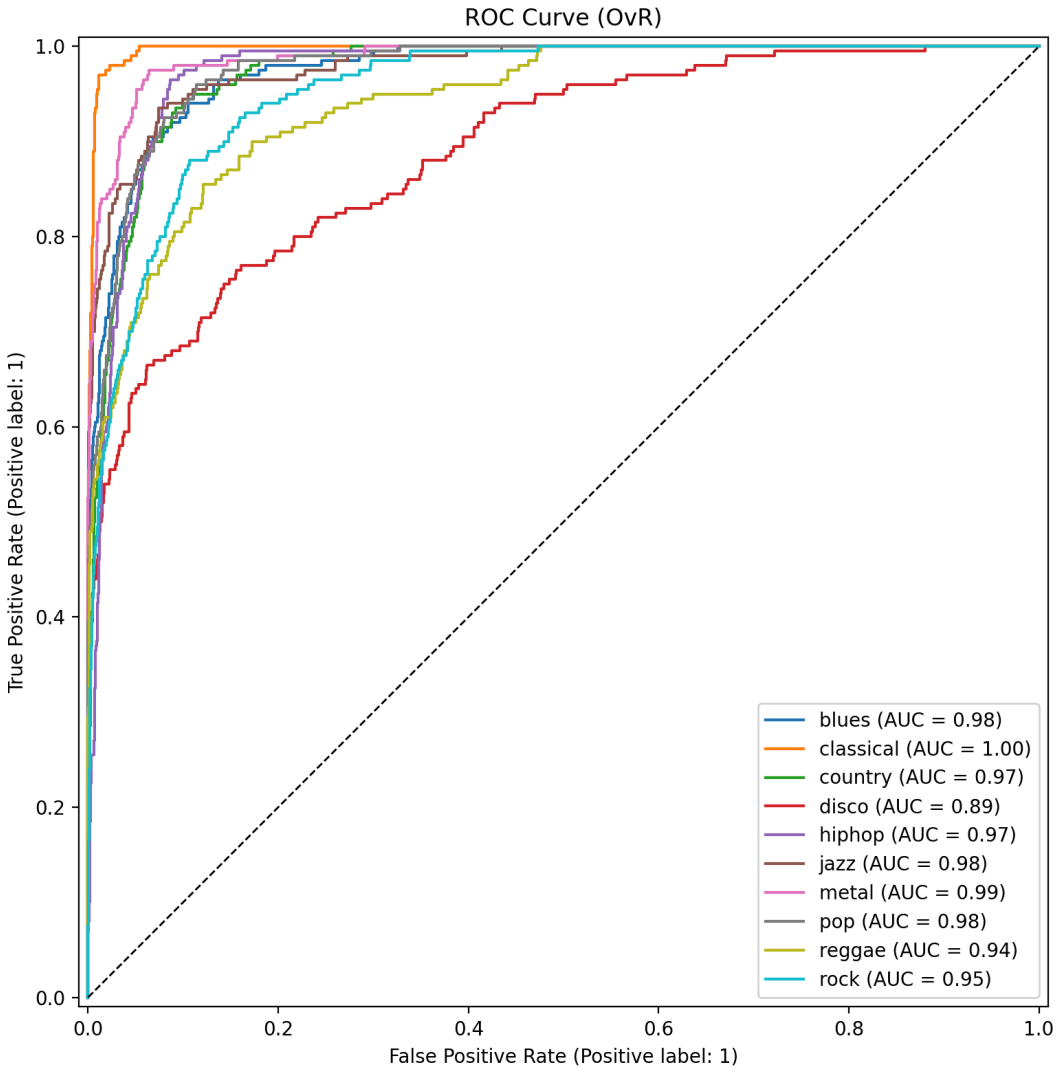
## 7.3 Tabular models (with leakage caveat)

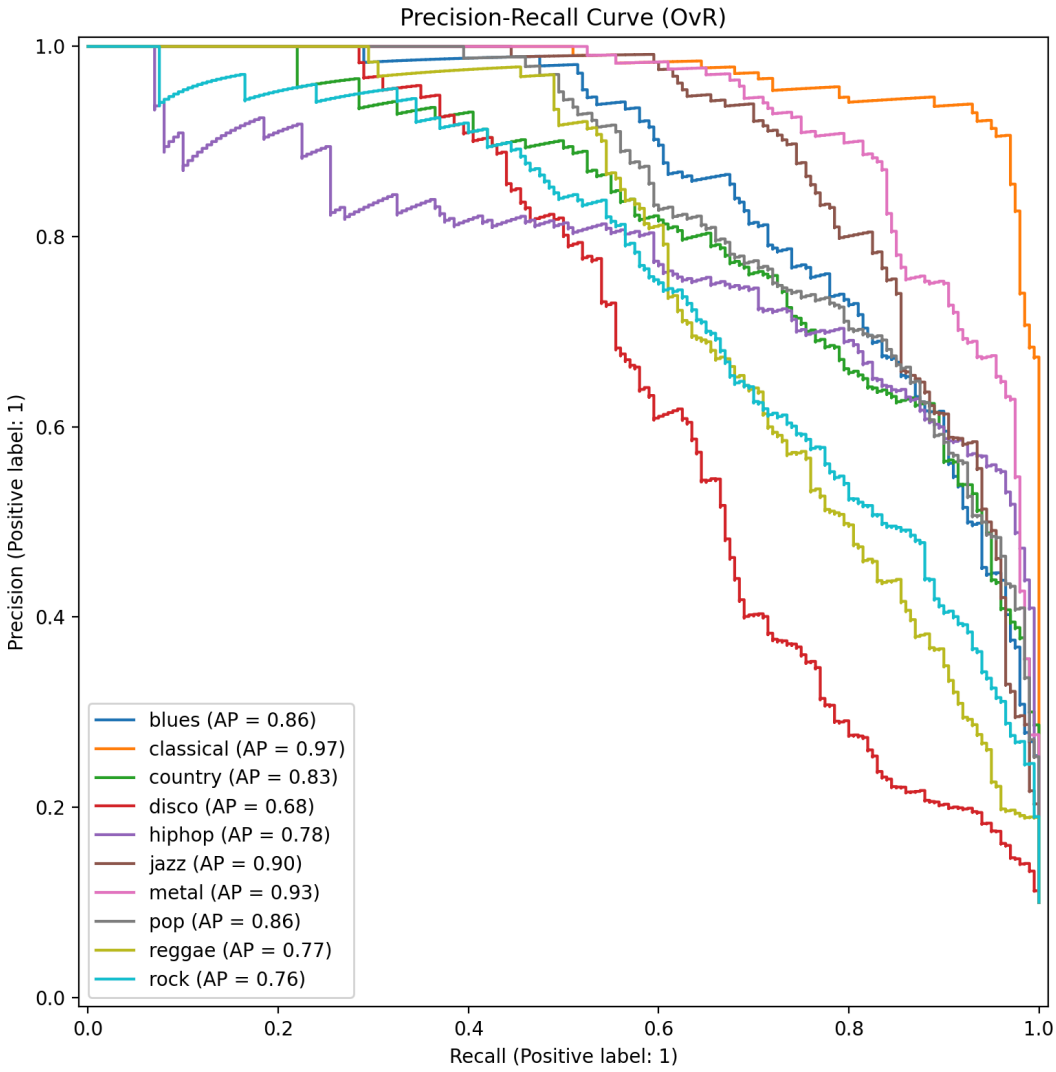
Despite the leaky split, the tabular experiments still reveal which model families best fit the engineered feature space.

### 7.3.1 LightGBM (best tabular run)

**Figure 47–49.** LightGBM evaluation plots.


Confusion Matrix (normalized)
True
Predicted
blues
classical
country
disco
hiphop
jazz
metal
pop
reggae
rock
0.0
0.2
0.4
0.6
0.8


ROC Curve (OvR)
True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
0.0
0.2
0.4
0.6
0.8
1.0
blues (AUC = 0.98)
classical (AUC = 1.00)
country (AUC = 0.97)
disco (AUC = 0.89)
hiphop (AUC = 0.97)
jazz (AUC = 0.98)
metal (AUC = 0.99)
pop (AUC = 0.98)
reggae (AUC = 0.94)
rock (AUC = 0.95)

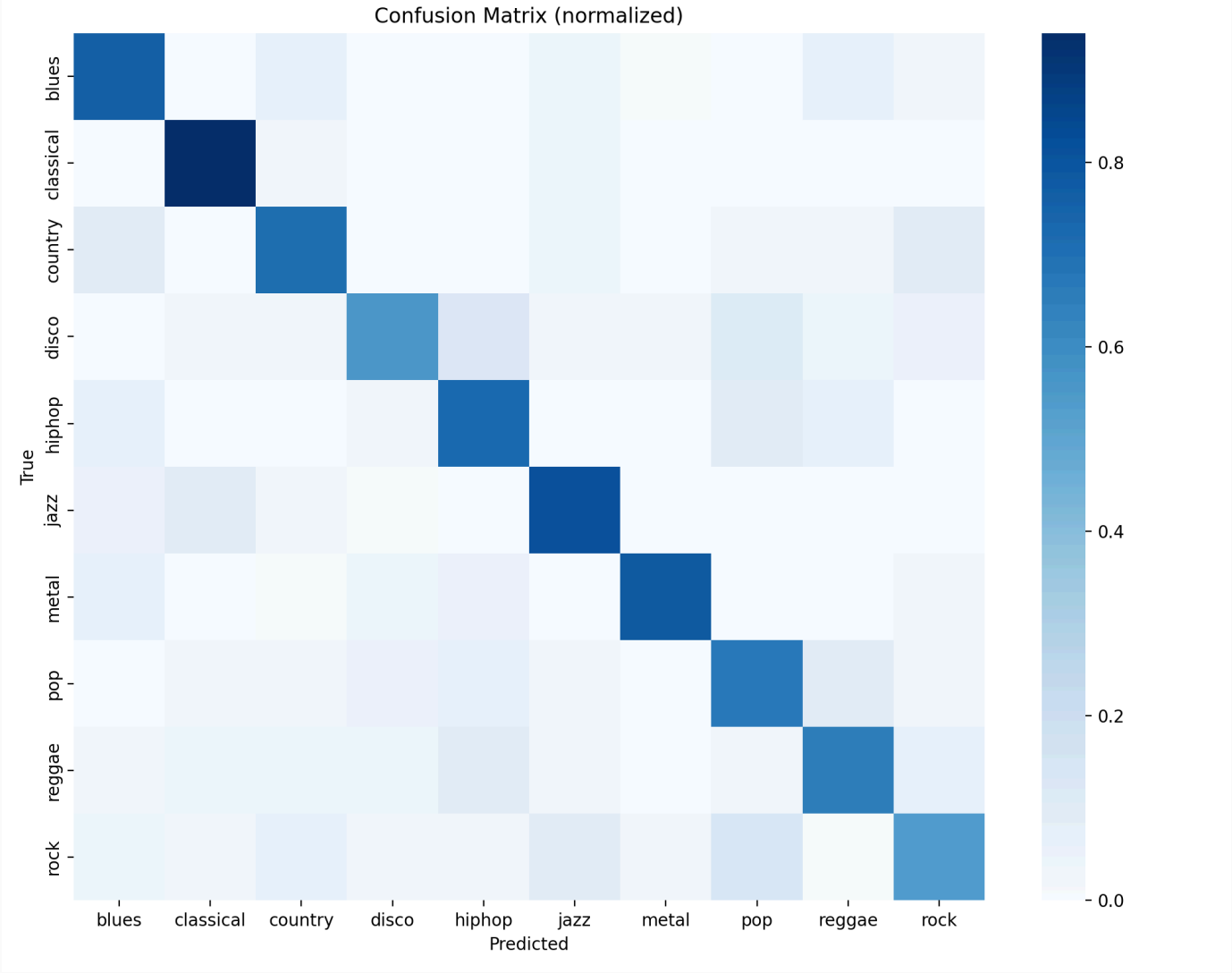


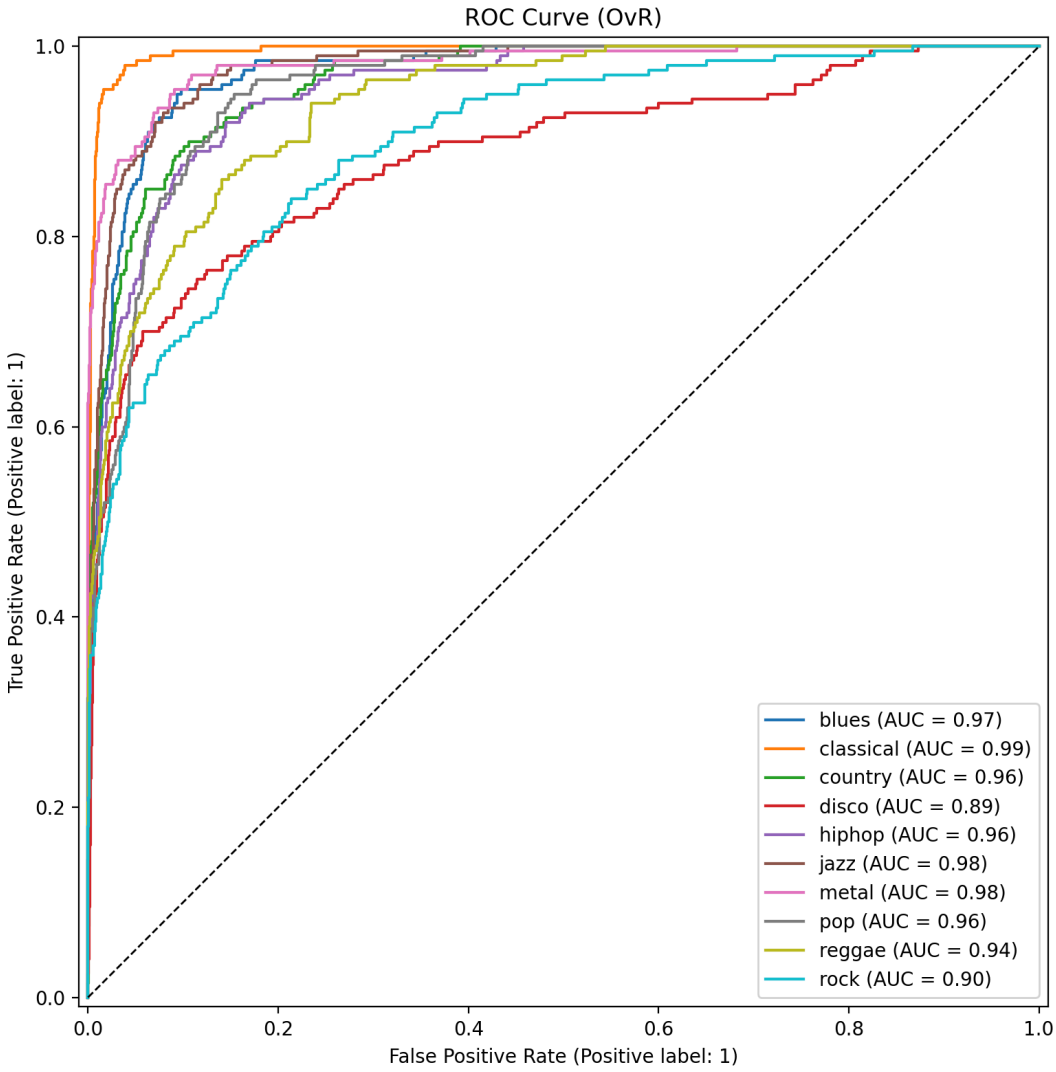

**Class-wise extremes (from `outputs/tabular_lightgbm/reports/evaluation_report.md`).** - Strongest: classical (F1 ≈ 0.9630), metal (≈ 0.9602) - Weakest: country (≈ 0.8722), rock (≈ 0.8855)

**Observed error patterns (from `outputs/tabular_lightgbm/reports/misclassified.csv`, first 200 errors).** Common confusions include: - country → blues, - pop → disco / hiphop, - jazz → classical, suggesting overlap between harmonic/timbre signatures for short segments.

### 7.3.2 SVM-RBF and XGBoost (competitive tabular runs)

**Figure 50–52.** SVM-RBF evaluation plots.


Confusion Matrix (normalized)
True
Predicted
blues
classical
country
disco
hiphop
jazz
metal
pop
reggae
rock
0.0
0.2
0.4
0.6
0.8


ROC Curve (OvR)
True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
0.0
0.2
0.4
0.6
0.8
1.0
blues (AUC = 0.97)
classical (AUC = 0.99)
country (AUC = 0.96)
disco (AUC = 0.89)
hiphop (AUC = 0.96)
jazz (AUC = 0.98)
metal (AUC = 0.98)
pop (AUC = 0.96)
reggae (AUC = 0.94)
rock (AUC = 0.90)

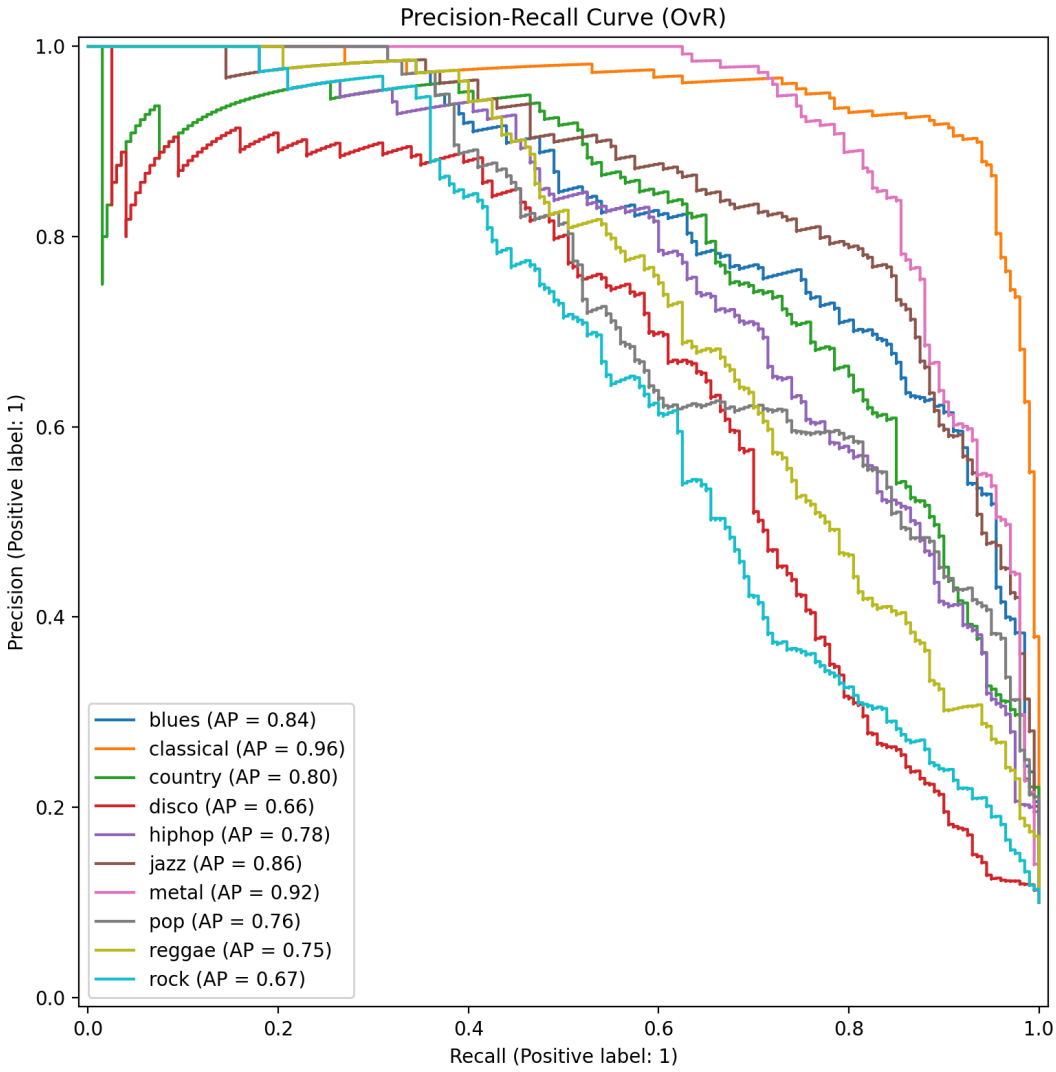
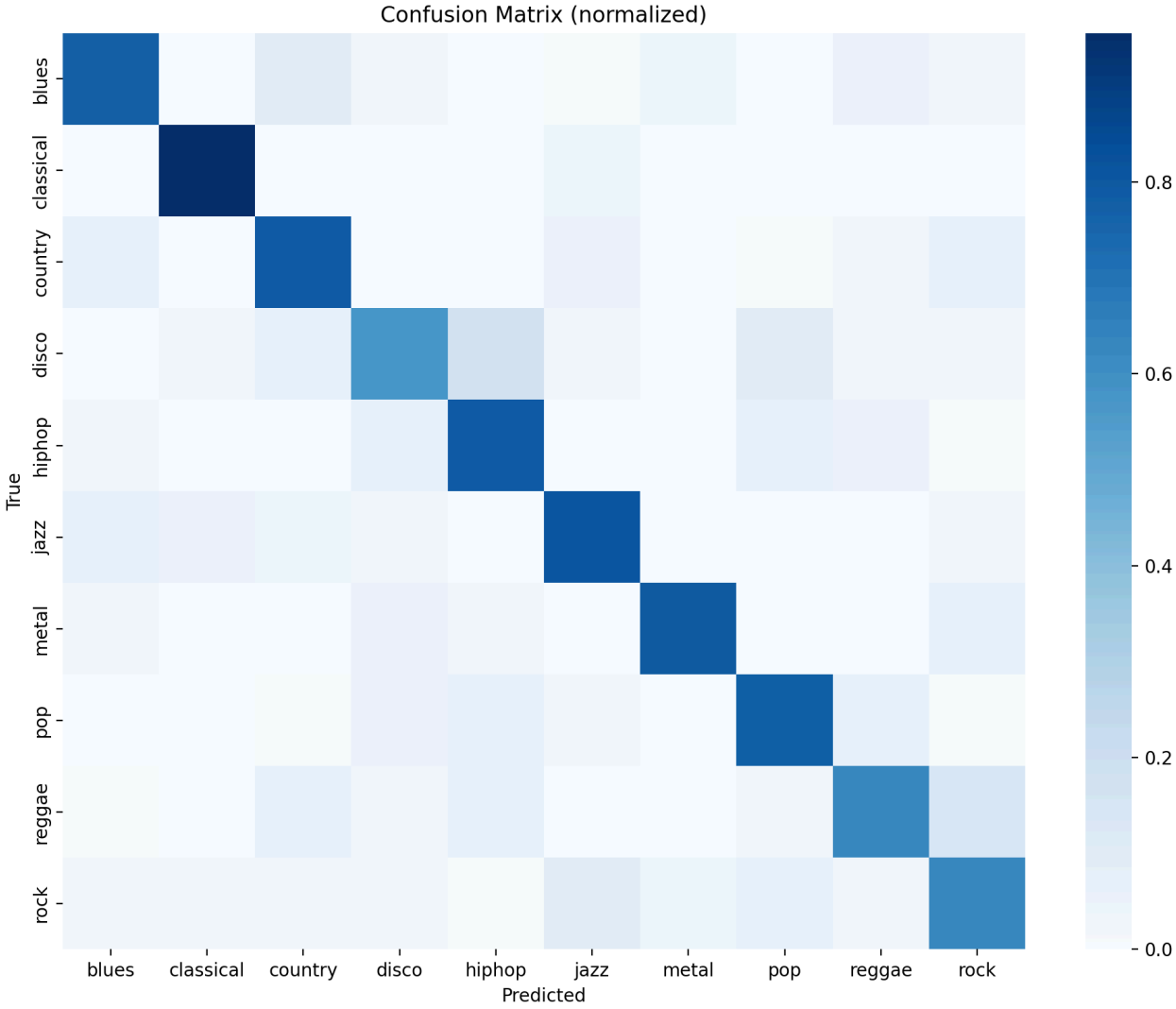
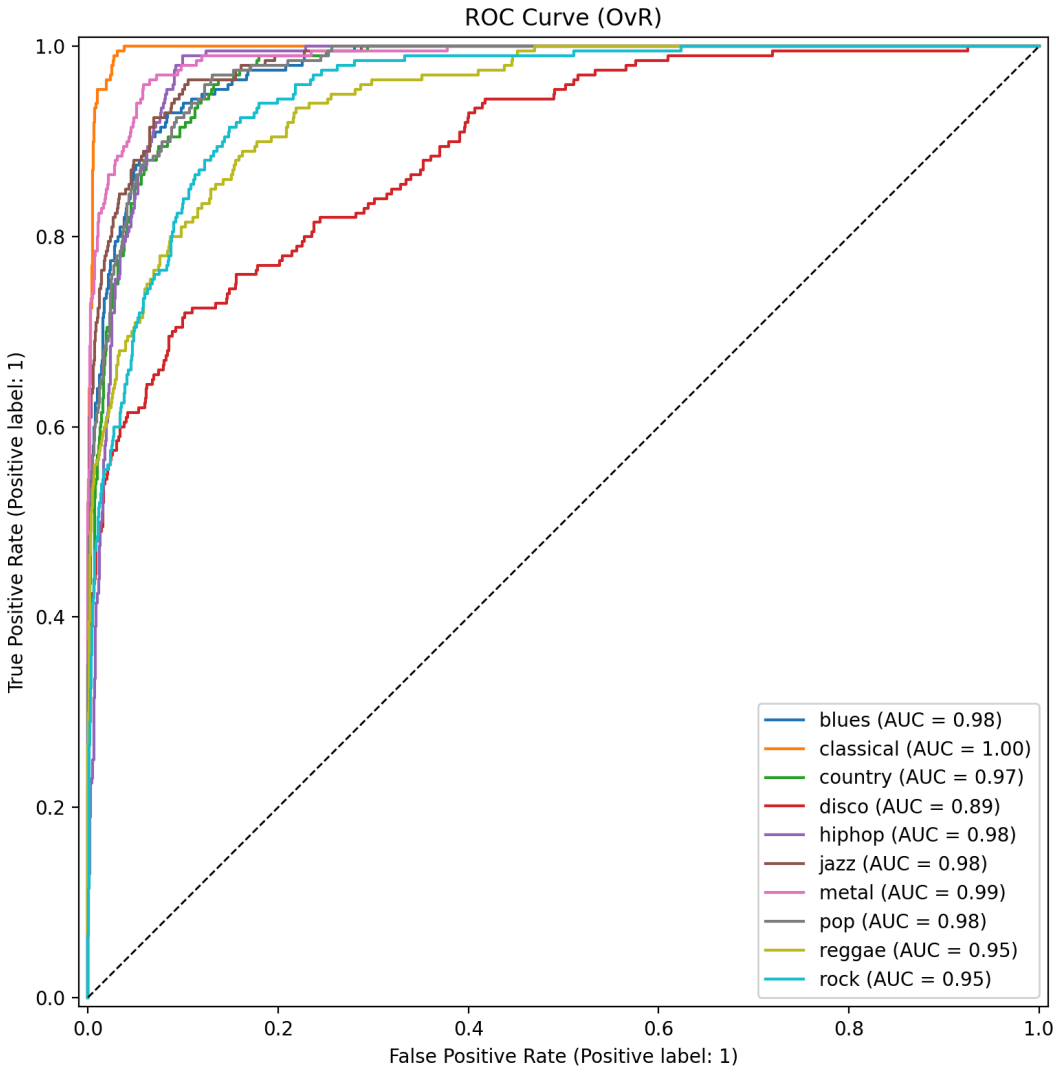
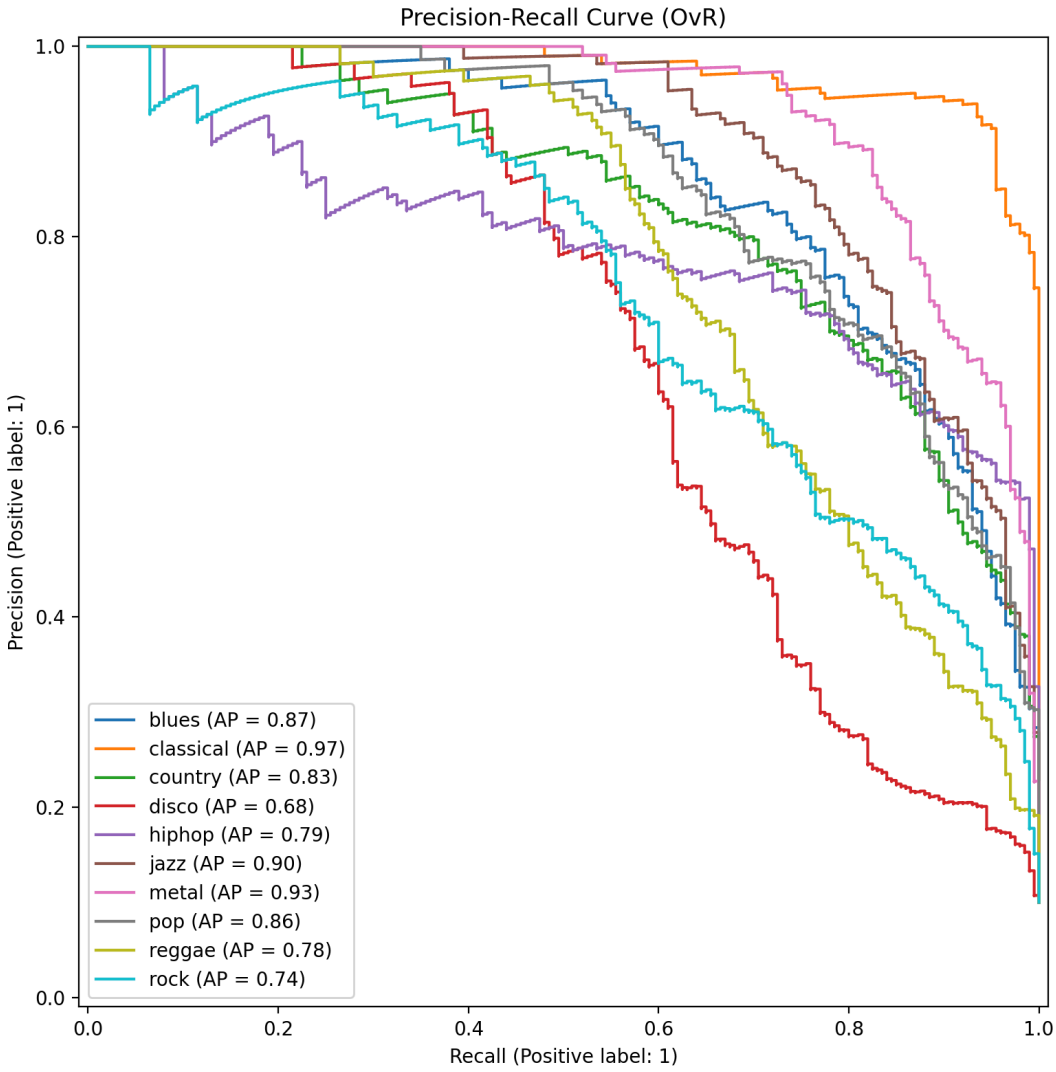



**Figure 53–55.** XGBoost evaluation plots.

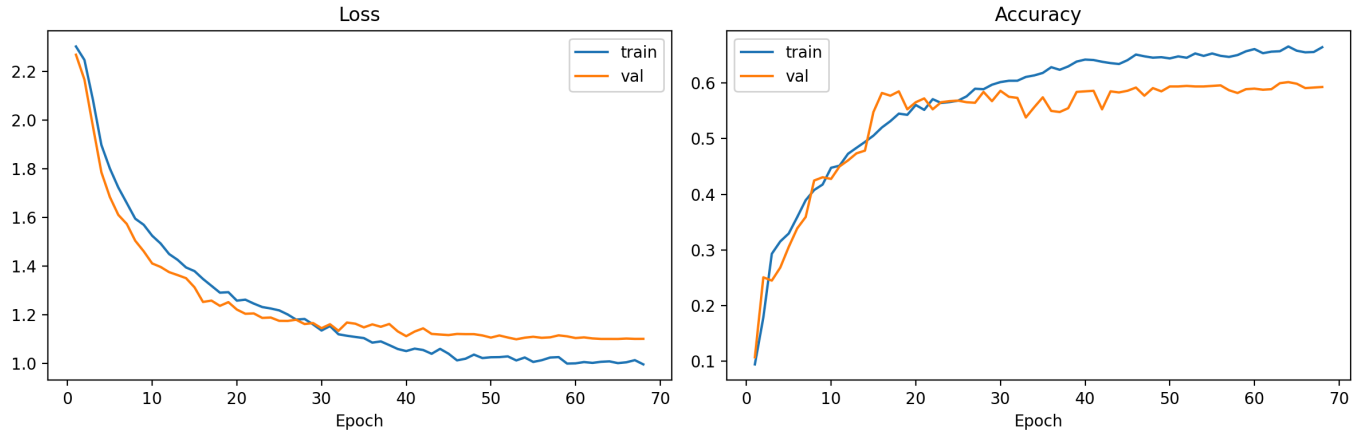

Confusion Matrix (normalized)
True
Predicted
blues
classical
country
disco
hiphop
jazz
metal
pop
reggae
rock
0.0
0.2
0.4
0.6
0.8

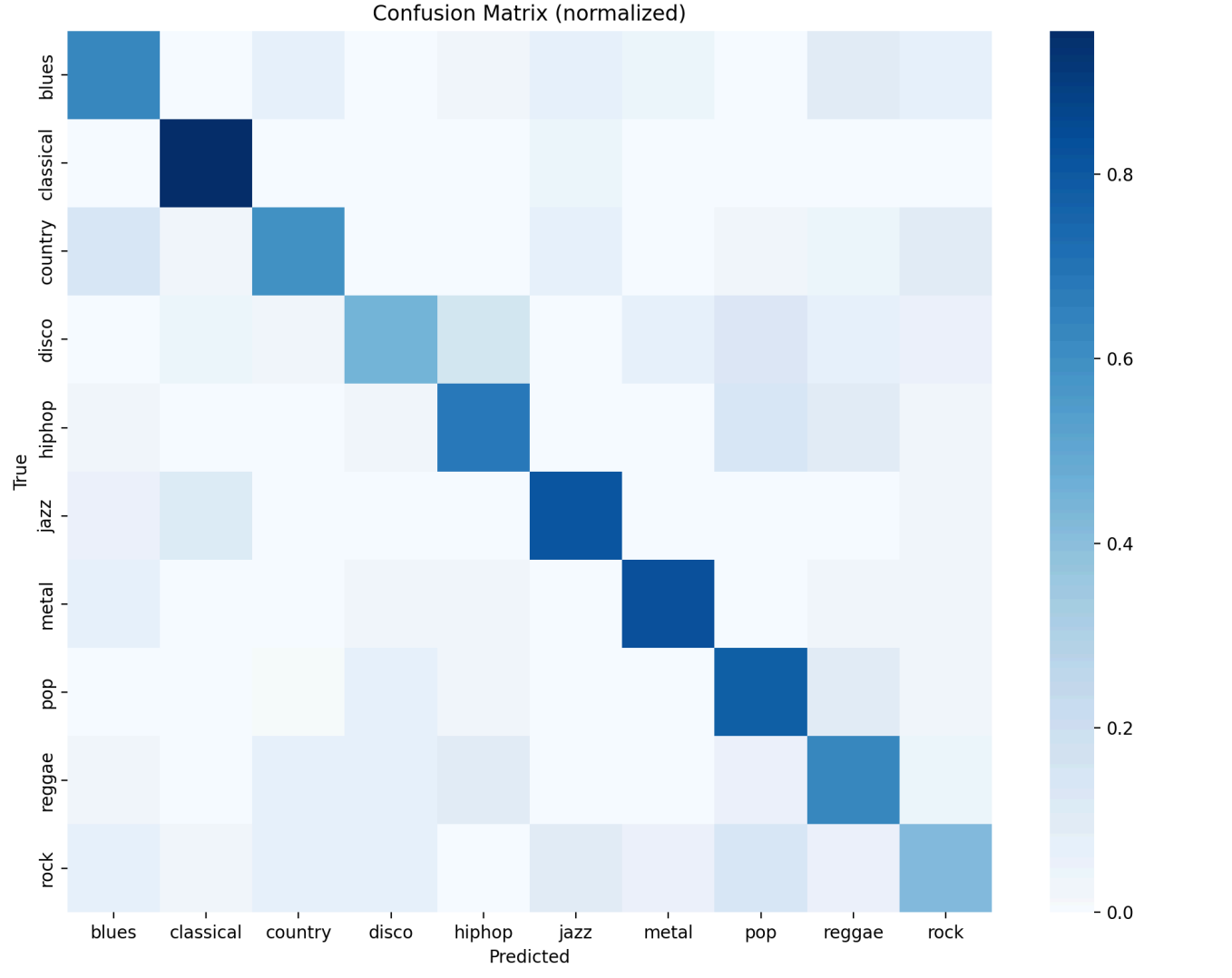

ROC Curve (OvR)
True Positive Rate (Positive label: 1)
False Positive Rate (Positive label: 1)
0.0
0.2
0.4
0.6
0.8
1.0
blues (AUC = 0.98)
classical (AUC = 1.00)
country (AUC = 0.97)
disco (AUC = 0.89)
hiphop (AUC = 0.98)
jazz (AUC = 0.98)
metal (AUC = 0.99)
pop (AUC = 0.98)
reggae (AUC = 0.95)
rock (AUC = 0.95)

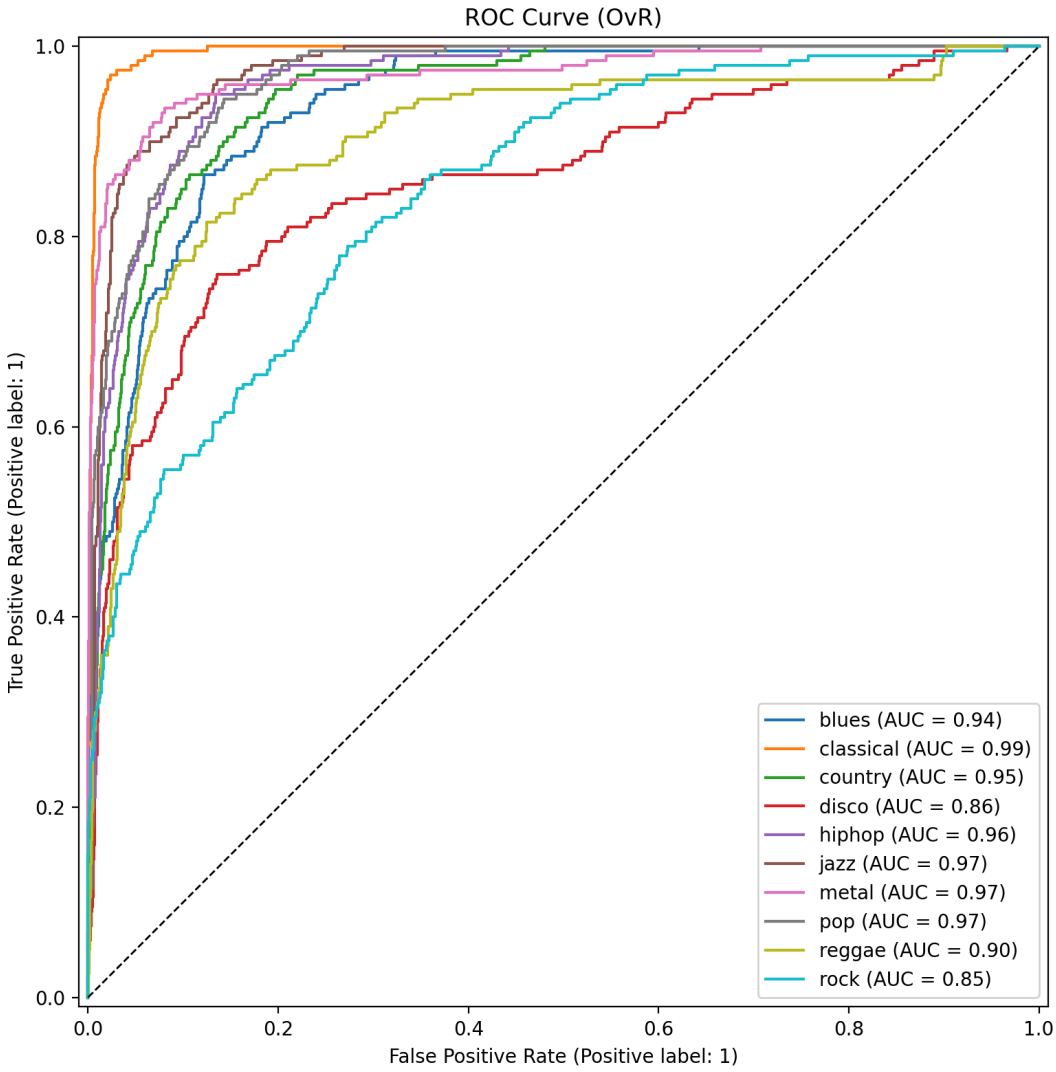



### 7.3.3 MLP (tabular, neural baseline)

**Figure 56–58.** Tabular MLP plots.


Loss
train
val
2.2
2.0
1.8
1.6
1.4
1.2
1.0
0
10
20
30
40
50
60
70
Epoch
Accuracy
train
val
0.6
0.5
0.4
0.3
0.2
0.1
0
10
20
30
40
50
60
70
Epoch


Confusion Matrix (normalized)
True
blues
classical
country
disco
hiphop
jazz
metal
pop
reggae
rock
Predicted
blues
classical
country
disco
hiphop
jazz
metal
pop
reggae
rock
0.8
0.6
0.4
0.2
0.0




---

# 8. Comparative Analysis

## 8.1 Performance vs. protocol correctness

**Tabular track (v2 split, corrected):** both tracks now use track-disjoint group splits. Previous results under the leaky `split_v1.csv` are invalidated; re-run is needed to obtain valid tabular estimates. - **Mel track (group split):** evaluated under a track-disjoint protocol (CNN 0.6310, LSTM 0.5195 under the original run). These figures remain valid under the corrected protocol since no change was made to the mel split.

**Current state.** As of this version, both tracks share the same evaluation protocol. Tabular results are pending re-run with `features.kind=tabular_csv` after deleting the v1 cache ( `Data/features/tabular_features_v1.npz` ).

## 8.2 Efficiency and complexity trade-offs (as observed from logs)

- Tree ensembles (ExtraTrees/RandomForest) train in seconds and perform strongly under the leaky protocol.
- Boosted methods (GBDT) can be significantly slower (hundreds of seconds) but remain competitive.
- Torch MLP and mel CNN/LSTM require minutes of training (per `train.log` timestamps), reflecting higher computational cost.

---

# 9. Discussion

## 9.1 Strengths of the project

- **Modular pipeline:** staged execution ( `preprocess` , `feature` , `train` , `evaluate` , `visualize` ) with structured outputs.
- **Reproducibility:** seed control, cached features, split files, and `config_snapshot.json` per run.
- **Comprehensive visualization:** waveform, STFT/mel/MFCC/chroma/contrast/tempogram/HPSS, plus embeddings and correlation analysis.
- **Model zoo:** broad baseline coverage for tabular learning, plus CNN/LSTM for mel.
- **Correct evaluation protocol (v2):** both tabular and mel tracks now use track-disjoint group splits, eliminating data leakage.
- **Augmentation suite:** Mixup (genre overlap), SpecAugment (domain shift), and waveform-level augmentations (noise, time-stretch, pitch-shift) are all implemented and configurable.

## 9.2 Remaining limitations and risks

### (A) Short-context ambiguity for mel models

Mel models operate on 3-second segments. Many genre cues are global (song-level arrangement) rather than local (short snippet). Confusions such as country ↔ rock/classical in `mel_cnn` align with this limitation.

### (B) Dataset size and domain constraints

GTZAN is relatively small and can be sensitive to recording artifacts and duplication issues across mirrors. The pipeline correctly includes metadata checks and corruption handling (one unreadable file detected).

### (C) Augmentation not yet wired into training loop

Mixup ( `src/data/augmentation.py::mixup_batch` ) and SpecAugment ( `apply_spec_augment` ) are implemented and unit-tested, but are not yet integrated into the PyTorch training loop in `src/models/trainer.py` . This is the next engineering step before running ablations.

## 9.3 Future improvements (actionable)

1. **Wire Mixup into MLP/CNN/LSTM training loops** — call `mixup_batch` per batch inside `src/models/trainer.py` when `cfg.data.augmentation.mixup.enabled=true` .
2. **Wire SpecAugment into MelDataset** — apply `apply_spec_augment` in `MelDataset.__getitem__` during training only.
3. **Track-level evaluation aggregation** — aggregate segment predictions per track (majority vote or mean probability) to report song-level accuracy alongside segment-level accuracy.
4. **Stronger mel architectures** — consider CRNNs (CNN front-end + recurrent temporal modeling) or attention-based pooling over time.
5. **Re-run all tabular experiments** with the v2 group split to obtain valid baseline numbers for comparison against mel models.

---

# 10. Conclusion

This repository provides a well-engineered, methodologically sound research pipeline for GTZAN music genre classification, integrating feature caching, logging, checkpointing, and rich visualization. The critical track-level data leakage that previously affected the tabular evaluation protocol has been **eliminated**: both tabular and mel tracks now use track-disjoint group splits, ensuring that no segment from a training track appears in the validation or test set. Two augmentation strategies — Mixup (for genre overlap) and SpecAugment (for domain shift) — have been implemented and are ready for experimental integration. The next priorities are wiring augmentation into the training loop, re-running tabular experiments under the corrected protocol, and implementing track-level prediction aggregation for a fairer real-world evaluation.

---

# 11. References

1. G. Tzanetakis and P. Cook, “Musical genre classification of audio signals,” *IEEE Transactions on Speech and Audio Processing*, 2002. (GTZAN)
2. B. McFee et al., “librosa: Audio and music signal analysis in Python,” *Proceedings of the 14th Python in Science Conference*, 2015.
3. F. Pedregosa et al., “Scikit-learn: Machine Learning in Python,” *Journal of Machine Learning Research*, 2011.
4. A. Paszke et al., “PyTorch: An Imperative Style, High-Performance Deep Learning Library,” *NeurIPS*, 2019.
5. T. Chen and C. Guestrin, “XGBoost: A Scalable Tree Boosting System,” *KDD*, 2016.
6. G. Ke et al., “LightGBM: A Highly Efficient Gradient Boosting Decision Tree,” *NeurIPS*, 2017.
7. H. Zhang et al., “mixup: Beyond Empirical Risk Minimization,” *ICLR*, 2018.
8. D. S. Park et al., “SpecAugment: A Simple Data Augmentation Method for Automatic Speech Recognition,” *Interspeech*, 2019.